

Mô hình sinh, style transfer và low-level vision (Generative Models and Low-Level Vision)

PHỤ THUỘC B/09-thiet-ke-ham-mat-mat (hướng đo khoảng cách phân phối — điều kiện tiên quyết thật sự của chương này); C/12-co-che-huan-luyen (huấn luyện ổn định, quy mô); C/13-kien-truc-backbone (U-Net, transformer); A/02-vat-ly-tao-anh (nhiều thật, dài sáng động).

TÓM TẮT

Chương này gom bốn chủ đề thường bị dạy rời nhau — mô hình sinh, chuyển phong cách, diffusion, và khôi phục ảnh — vì chúng chia chung đúng một câu hỏi: khi nhiều đầu ra cùng hợp lệ, cái gì quyết định chọn cái nào? Bốn họ mô hình sinh là bốn câu trả lời, chuyển phong cách là câu trả lời viết thẳng lên pixel, diffusion là câu trả lời hiện đang thắng, và low-level vision là nơi câu hỏi trở thành vấn đề đúng sai chứ không còn là thẩm mỹ. Đọc xong bạn nhìn một mô hình sinh là đọc ra kiểu thất bại của nó trước khi chạy. Nếu chỉ nhớ một điều: chất lượng đầu ra khác nhau là vì *định nghĩa* “giống dữ liệu thật” khác nhau, không phải vì mô hình này thông minh hơn mô hình kia — và mọi cơ chế làm ảnh “đẹp hơn” đều mua độ trung thành bằng độ đa dạng.

KIẾN THỨC CẦN CÓ TRƯỚC

- **mô hình sinh (generative model)** — mô hình học *phân phối* của dữ liệu, để có thể lấy mẫu ra dữ liệu mới; khác với mô hình phân biệt, vốn chỉ học ánh xạ từ đầu vào sang nhãn.
- **phân phối dữ liệu (data distribution)** — cách nghĩ về tập ảnh thật như các mẫu rút ra từ một phân phối xác suất trên không gian ảnh; “sinh ảnh mới” nghĩa là rút thêm một mẫu từ phân phối đó.
- **hàm mất mát (loss function)** — đại lượng số mà quá trình huấn luyện cực tiểu hoá. Toàn chương này là một minh hoạ kéo dài cho nguyên tắc: chọn hàm mất mát là chọn trước kiểu sai lầm mà mô hình sẽ mắc.
- **không gian ẩn (latent space)** — không gian chiều thấp mà mô hình dùng để mã hoá một ảnh; nó vừa là nơi “ý nghĩa” được sắp xếp, vừa là mẹo giúp diffusion chạy được trên GPU thường.
- **style transfer (chuyển phong cách)** — bài toán giữ nội dung của một ảnh nhưng khoác lên nó “chất” của một ảnh khác — nét cọ, bảng màu, kết cấu.
- **ill-posed (đặt không chính)** — bài toán có nhiều nghiệm cùng hợp lệ vì thông tin đã bị xoá; mọi bài toán khôi phục ảnh đều thuộc loại này.
- **FID / LPIPS / PSNR** — ba chỉ số đánh giá ở ba tầng khác nhau: FID so hai *phân phối* ảnh, LPIPS so hai ảnh trong không gian đặc trưng học được, PSNR so hai ảnh theo từng pixel. Mục 17.4 nói rõ mỗi cái mù với gì.

CÂU HỎI MỞ ĐẦU

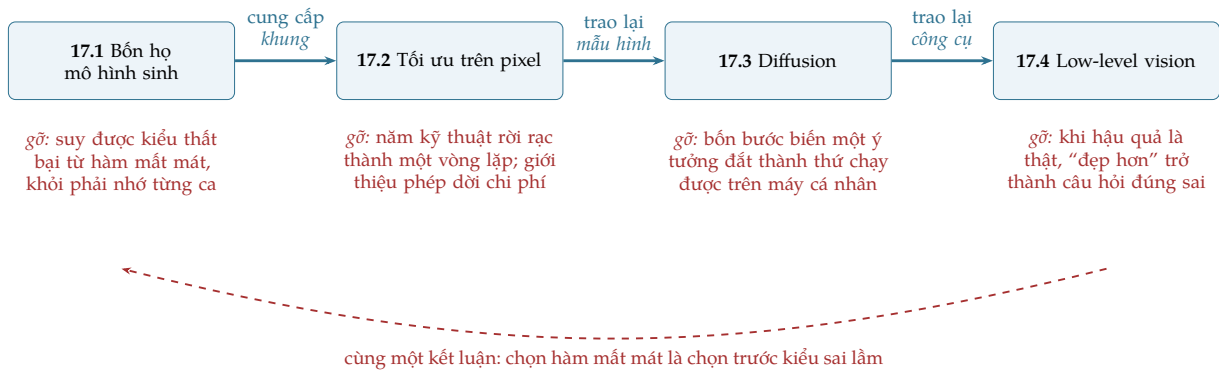
- Q1. Vì sao mô hình tối ưu MSE *luôn* cho ảnh mờ, và vì sao đó là một kết quả toán học chứ không phải lỗi cài đặt?
- Q2. VAE hỏng vì mờ, GAN hỏng vì nghèo — hai triệu chứng trái ngược nhau. Vì sao chúng lại là hai mặt của cùng một quyết định thiết kế?

- Q3.** Vì sao diffusion, thứ phải chạy mạng hàng chục tới hàng trăm lần cho một ảnh, lại đánh bại GAN vốn chỉ cần một lượt xuôi?
- Q4.** Vì sao chỉnh một pipeline sinh ảnh tới lúc trông “đẹp nhất” lại là cách chắc chắn nhất để làm hỏng nó khi dùng đầu ra làm dữ liệu huấn luyện?

Vì sao chương này chia làm bốn mục

Bốn mục là bốn tầng của cùng một câu hỏi, và thứ tự là thứ tự phụ thuộc.

Hình 1.1 vẽ bốn mục cùng nút thắt mà mỗi mục gỡ.



Hình 1.1. Bốn mục của chương, xếp theo thứ tự phụ thuộc chứ không theo mức độ phổ biến. Mục 17.2 nằm giữa lý thuyết và diffusion không phải ngẫu nhiên: nó là ví dụ sạch nhất cho việc hàm mục tiêu quyết định tất cả, và nó giới thiệu mẫu hình “dời chi phí sang lúc huấn luyện” mà mục 17.3 dùng lại nguyên vẹn. Mỗi tên dứt là kết luận chung mà cả bốn mục đều dẫn tới.

Mục thứ nhất dựng khung: bốn họ mô hình sinh là bốn định nghĩa khác nhau của cụm từ “giống dữ liệu thật”. Khung này quan trọng hơn bất kỳ chi tiết kiến trúc nào, vì nó cho phép suy ra kiểu thất bại từ hàm mất mát — ảnh mờ, mất mode, hay chậm — thay vì phải ghi nhớ từng trường hợp.

Mục thứ hai tách riêng một ý tưởng cốt ngang: đóng băng mạng, coi ảnh là biến, hạ gradient trên chính pixel. Nó được đặt ở đây, giữa lý thuyết và diffusion, vì hai lý do: nó là ví dụ sạch nhất về việc hàm mục tiêu quyết định tất cả (đổi một dòng là được một kỹ thuật khác), và nó giới thiệu mẫu hình “dời chi phí từ lúc chạy sang lúc huấn luyện” mà mục ba sẽ dùng lại nguyên vẹn.

Mục thứ ba là họ đang thắng. Nó không chỉ trình bày nguyên lý mà đi qua bốn bước kỹ thuật đã biến diffusion từ một ý tưởng thành sản phẩm chạy được trên máy cá nhân — và mỗi bước là một đánh đổi cụ thể mà người dùng phải chỉnh bằng tay.

Mục thứ tư đưa cùng bộ khái niệm sang một bối cảnh mà hậu quả là thật: khôi phục ảnh. Ở đây “sắc nét hơn” đồng nghĩa với “bịa nhiều hơn”, và điều đó không phải một lời than phiền mà là một kết quả có chứng minh. Mục này đặt cuối vì nó cần cả ba mục trước để nói cho đủ.

Bản đồ chương 17 — Mô hình sinh, style transfer và low-level vision

TÓM TẮT

Chương 17 gồm bốn mục, xếp theo thứ tự phụ thuộc chứ không theo mức độ phổ biến: khung bốn họ mô hình sinh, ý tưởng tối ưu trên pixel, diffusion, rồi khôi phục ảnh. Bản đồ này nói mục nào cốt lõi, mục nào tra khi cần, và trình tự đọc tùy mục tiêu của bạn. Nếu chỉ nhớ một điều: đọc mục 17.1 trước — nó cho phép *suy ra* kiểu thất bại của một mô hình từ hàm mất mát, và ba mục còn lại đều dùng lại khả năng đó.

KIẾN THỨC CẦN CÓ TRƯỚC

- **mô hình sinh (generative model)** — mô hình học phân phối dữ liệu để lấy mẫu ra dữ liệu mới, khác với mô hình dự đoán nhãn.
- **diffusion** — họ mô hình sinh bằng cách học khử nhiễu dần, từ nhiễu trắng về ảnh; hiện là họ thống trị.
- **style transfer** — giữ nội dung của ảnh này, khoác “chất” của ảnh kia.
- **low-level vision** — nhóm bài toán ảnh sang ảnh: khử nhiễu, siêu phân giải, khử nhoè, HDR.

Bốn mục và vai trò của từng mục

- **17.1 — Bốn họ và cuộc đánh đổi ba chiều (cốt lõi).** Autoregressive, VAE, GAN, diffusion là bốn định nghĩa của “giống dữ liệu thật”. Đây là khung để suy ra kiểu thất bại, và là mục đắt giá nhất của cả chương.
- **17.2 — Tối ưu trên pixel thay vì trên trọng số (cốt lõi về ý tưởng, tham khảo về chi tiết style transfer).** Một vòng lặp mười dòng cho ra năm kỹ thuật; ma trận Gram; mẫu hình dờ chi phí sang lúc huấn luyện.
- **17.3 — Diffusion và các bước làm cho nó khả thi (cốt lõi).** Quá trình thuận/ngược, score function, bộ lấy mẫu, dẫn hướng, khuếch tán trong không gian ẩn, ControlNet.
- **17.4 — Low-level vision và trade-off không thể tránh (cốt lõi với người làm hệ thống thật).** Ill-posed, đánh đổi perceptual-distortion, các chỉ số và chỗ mù của từng cái, dữ liệu sinh dùng để huấn luyện.

Trình tự đọc

1. **Mục tiêu là hiểu nguyên lý:** 17.1 → 17.3 → 17.4, bỏ qua 17.2 ở lần đọc đầu.
2. **Mục tiêu là dùng được công cụ sinh ảnh:** 17.1 (chỉ phần bảng và tam giác) → 17.3 toàn bộ → 17.4 phần đánh giá.
3. **Mục tiêu là làm hệ thống thật có khôi phục ảnh:** 17.4 trước, rồi quay lên 17.1 để hiểu vì sao “mờ” là nghiệm đúng.
4. **Khối Thực hành** nằm ở file chương, sau bốn mục.

Chương này phụ thuộc gì và được dùng lại ở đâu

- **Phụ thuộc lên trên:** B/09-thiet-ke-ham-mat-mat (hai hướng của KL — điều kiện tiên quyết thật sự); C/12-co-che-huan-luyen; C/13-kien-truc-backbone (U-Net, transformer); A/02-vat-ly-tao-anh (nhiều thật); C/16-thi-giac-3d (vai trò của prior trong bài toán thiếu ràng buộc).
- **Được dùng lại ở dưới:** D/20-toi-uu-va-nen-mo-hinh (dời chi phí sang lúc huấn luyện, chung cất); C/18-pretraining-va-foundation (nguồn giám sát và dữ liệu sinh);

E/25-do-tin-cay-va-van-hanh (hồng im lặng, trực quan hoá và các phép kiểm tra tính đúng đắn).

1.1 Bốn họ và cuộc đánh đổi ba chiều (Four Families of Generative Models)

PHỤ THUỘC B/09-thiet-ke-ham-mat-mat (hướng đo khoảng cách phân phối, forward đối lại reverse KL); A/01-toan-toi-thieu/03-xac-suat-va-thong-tin (likelihood, entropy tương đối); C/13-kien-truc-backbone (U-Net, transformer).

TÓM TẮT

Bốn họ mô hình sinh — *autoregressive*, VAE, GAN, *diffusion* — là bốn định nghĩa khác nhau của cụm từ “giống dữ liệu thật”. Chất lượng đầu ra khác nhau chính vì định nghĩa khác nhau, không phải vì mô hình này thông minh hơn mô hình kia. Đọc xong bạn nhìn một mô hình sinh là đọc ra ngay nó đang tối ưu đại lượng gì. Từ đó suy được nó sẽ mờ, sẽ thiếu đa dạng, hay sẽ chậm. Và bạn biết mình đang mua cái gì bằng cái gì. Nếu chỉ nhớ một điều: ảnh mờ của VAE và *mode collapse* của GAN không phải hai lỗi kỹ thuật khác nhau mà là hai mặt của cùng một lựa chọn — đo khoảng cách phân phối theo hướng nào.

KIẾN THỨC CẦN CÓ TRƯỚC

- **likelihood (độ hợp lý)** — xác suất mà một mô hình gán cho dữ liệu đã quan sát; “cực đại hoá likelihood” là chỉnh tham số sao cho mô hình thấy dữ liệu thật là chuyện bình thường chứ không phải chuyện lạ.
- **KL divergence (phân kỳ Kullback–Leibler)** — một cách đo khoảng cách giữa hai phân phối; nó *không đối xứng*, và chính sự không đối xứng đó sinh ra khác biệt giữa “mờ” và “mất mode”.
- **mode (mode của phân phối)** — một vùng tập trung nhiều xác suất — ví dụ “ảnh mèo” và “ảnh chó” là hai mode. Phủ hết mode nghĩa là sinh ra được cả hai loại.
- **mode collapse** — hiện tượng mô hình chỉ sinh ra một phần nhỏ của phân phối (chỉ vài kiểu ảnh) trong khi vẫn đạt điểm cao theo tiêu chí huấn luyện.
- **biến ẩn (latent variable)** — vectơ chiều thấp mà mô hình dùng làm “bản tóm tắt” của một mẫu; sinh ảnh là lấy ngẫu nhiên một vectơ như vậy rồi giải mã.
- **discriminator (bộ phân biệt)** — mạng thứ hai trong GAN, có việc duy nhất là phân biệt mẫu thật với mẫu do mô hình sinh; nó đóng vai một hàm mất mát *học được*.
- **score matching** — cách huấn luyện bằng việc khớp gradient của log mật độ thay vì khớp mật độ; đây là nền toán của diffusion.

A. Đặt vấn đề

Một mô hình sinh phải làm được một việc nghe rất mơ hồ: tạo ra mẫu mới “giống” dữ liệu huấn luyện nhưng không phải bản sao của nó. Muốn tối ưu bất cứ thứ gì thì phải viết “giống” thành một con số, và đây là chỗ bốn họ mô hình rẽ nhánh — chúng viết ra bốn con số khác nhau. Autoregressive đo bằng **độ hợp lý** (*likelihood*) chính xác của từng pixel theo thứ tự; VAE đo bằng một cận dưới của likelihood; GAN đo bằng “một bộ phân biệt học được không tách nổi mẫu giả khỏi mẫu thật”; diffusion đo bằng mức khớp của *score* ở mọi mức nhiễu. Bốn định nghĩa, bốn hình dạng lỗi.

Điều làm chủ đề này đáng học kỹ là các chế độ lỗi có thể *suy ra* chứ không phải ghi nhớ. Khi biết một mô hình tối ưu trung bình bình phương theo pixel, bạn dự đoán được nó cho ảnh mờ trước khi chạy nó; khi biết một mô hình tối ưu một hàm mục tiêu dạng trò chơi hai người, bạn dự đoán được nó không hội tụ theo nghĩa thông thường.

Học mục này thì làm được gì mà trước đó không làm được? Đọc một bài báo mô hình sinh và trả lời trong ba mươi giây: mô hình này sẽ mờ, thiếu đa dạng, hay chậm — và vì sao bắt buộc phải như vậy. Sau đó chọn được họ mô hình cho một bài toán cụ thể thay vì chọn theo cái đang nổi.

TRỰC GIÁC

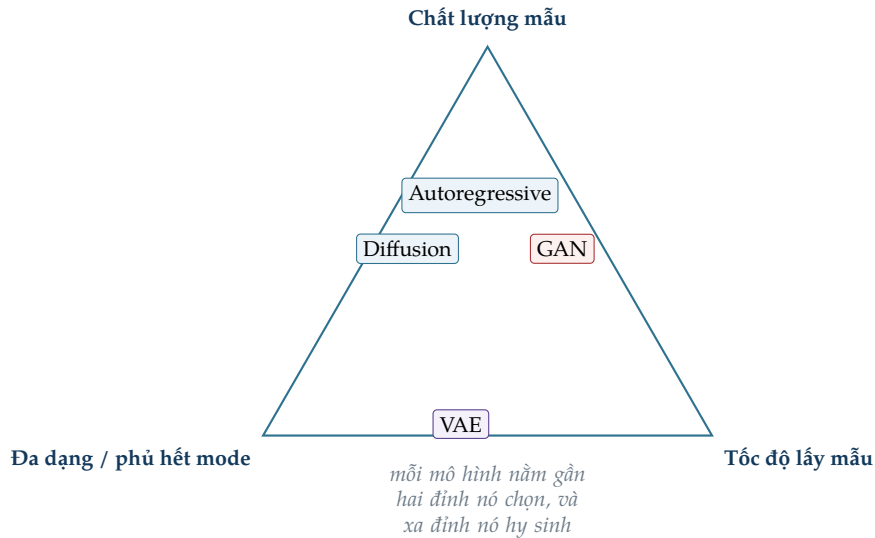
Bốn họ giống bốn cách chấm điểm một người làm tranh giả. *Autoregressive* bắt vẽ từng nét theo đúng xác suất của nét tiếp theo: chấm được điểm rất chính xác, nhưng vẽ chậm khủng khiếp vì mỗi nét phải chờ nét trước. *VAE* bắt nén bức tranh thành vài chục con số rồi giải nén: nhanh, nhưng phần giải nén trả về *trung bình* của mọi bức tranh khớp với mấy con số đó, nên nét bị nhoè. *GAN* thuê một giám định viên và chỉ đòi lừa được ông ta: nét rất sắc vì giám định viên soi nét — nhưng nếu ông ta chỉ quen soi hoa văn ở góc trái thì kẻ làm giả sẽ chỉ học vẽ giỏi đúng cái góc đó. *Diffusion* bắt đầu từ một tờ giấy đầy nhiễu và tẩy dần từng nấc, mỗi nấc chỉ phải đoán lùi một chút. Cách này chậm vì nhiều nấc. Bù lại mỗi nấc là một bài dễ, nên nó không bao giờ “vẽ hỏng”.

B. Bốn họ đặt cạnh nhau

Bảng 1.1. Bốn họ mô hình sinh, đọc theo định nghĩa của “giống”. Cột cuối suy ra được từ cột thứ hai — đó là điểm của cả bảng.

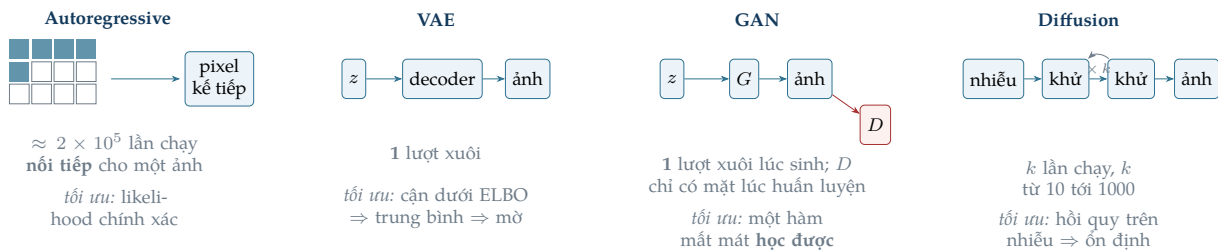
Họ	Định nghĩa “giống”	Được	Mất / hỏng khi
Auto-regressive (PixelRN-N/CNN)	Likelihood chính xác, phân rã theo quy tắc chuỗi	Tính được likelihood; huấn luyện ổn định	Sinh cực chậm vì tuần tự theo pixel; lệch giữa lúc huấn luyện và lúc sinh
VAE	Cận dưới của likelihood (ELBO)	Nhanh; có không gian ẩn có nghĩa, nội suy được	Ảnh mờ — hệ quả trực tiếp của loss trung bình; <i>posterior collapse</i>
GAN	“Một discriminator học được không phân biệt nổi”	Ảnh sắc nét ở cùng ngân sách tính toán	Huấn luyện bất ổn; mode collapse; không có likelihood để chẩn đoán
Diffusion	Khớp score của phân phối ở mọi mức nhiễu	Chất lượng và đa dạng cao; huấn luyện ổn định như hồi quy	Sinh chậm vì nhiều bước; nhạy với lựa chọn bộ lấy mẫu

Ba đại lượng trong cột ba và bốn không độc lập nhau. Trong tài liệu, người ta hay gọi tình thế này là *thế lưỡng nan ba chiều* của mô hình sinh: chất lượng mẫu, độ phủ hết các mode, và tốc độ lấy mẫu — ba thứ mà cho tới nay chưa họ nào đạt đồng thời cả ba ở mức tốt nhất. Cần nói rõ để trung thực: đây là một *cách gọi tên* một quan sát thực nghiệm lặp lại, không phải một định lý; nó khác hẳn về địa vị với đánh đổi perceptual-distortion ở mục 17.4, vốn là một kết quả có chứng minh. Hình 1.2 đặt bốn họ lên tam giác đó.



Hình 1.2. Thế lưỡng nan ba chiều. GAN mua chất lượng và tốc độ bằng độ phủ mode; VAE mua tốc độ và độ phủ bằng chất lượng; diffusion và autoregressive mua chất lượng cùng độ phủ bằng tốc độ. Toàn bộ tiến bộ kỹ thuật của mảng này — *sampler distillation, flow matching, latent diffusion* — là các nỗ lực kéo một điểm về gần tâm hơn.

Hình 1.3 bổ sung cho tam giác một thứ mà tam giác không nói được: *cơ chế* sinh ra mỗi vị trí trên đó.



Hình 1.3. Bốn cơ chế lấy mẫu, đặt cạnh nhau để thấy chi phí nằm ở đâu. Hai họ ở giữa sinh ảnh trong một lượt xuôi duy nhất; hai họ ở hai đầu phải chạy mạng nhiều lần, và đó chính là đỉnh “tốc độ lấy mẫu” mà chúng hy sinh trong Hình 1.2. Điểm đáng chú ý nhất là bộ phân biệt D của GAN: nó *không* tham gia lúc sinh, nó chỉ tồn tại lúc huấn luyện — vai trò của nó là một hàm mất mát học được, không phải một khối tính toán.

C. Vì sao họ sau ra đời

I MẠCH 1 — bốn định nghĩa của “giống dữ liệu thật”

- 1 Vấn đề** muốn mô hình hoá một phân phối trên không gian ảnh, mà không gian đó quá lớn để đếm.
- Giải pháp** phân rã theo quy tắc chuỗi: $p(x) = \prod_i p(x_i \mid x_{<i})$, rồi học từng thừa số bằng một mạng.
- Giả định** có một thứ tự tự nhiên trên pixel (thường là quét từ trái sang phải, trên xuống dưới — một giả định đã sai ngay từ đầu với ảnh).
- Hệ quả** likelihood tính được chính xác, nên có thể so sánh mô hình bằng một con số có ý nghĩa.
- Giá phải trả** lấy mẫu là tuần tự: một ảnh 256×256 màu cần 196 608 lần chạy mạng nối tiếp nhau; ở 1 ms mỗi lần thì hơn ba phút cho *một* ảnh.

- 2 Vấn đề** sinh tuần tự quá chậm để dùng được.
- Giải pháp** đưa vào một biến ẩn z chiều thấp, học đồng thời bộ mã hoá và bộ giải mã, tối ưu cận dưới ELBO [1]; lấy mẫu chỉ còn một lượt xuôi.
- Hệ quả** thêm một phần thưởng ngoài dự tính: không gian ẩn có cấu trúc, nội suy giữa hai điểm cho ra biến đổi mượt.
- Giá phải trả** bộ giải mã Gauss làm cho phần tái tạo của ELBO trở thành sai số bình phương trung bình theo pixel. Mà nghiệm tối ưu của bình phương trung bình là *kỳ vọng có điều kiện*, tức trung bình của mọi ảnh hợp lệ. Trung bình của nhiều ảnh sắc nét lệch nhau vài pixel là một ảnh mờ.
- **hệ quả của C/17/04** — cùng phép tính này quay lại ở khôi phục ảnh, nơi “mờ” trở thành nghiệm tối ưu PSNR và là một nửa của đánh đổi perceptual-distortion.
- Vấn đề mới** ảnh mờ không sửa được bằng cách huấn luyện lâu hơn; nó là nghiệm đúng của bài toán đã đặt ra.
- 3 Vấn đề** muốn ảnh sắc thì phải bỏ hàm mất mát trung bình, nhưng ta không biết viết công thức nào cho “trông thật”.
- Giải pháp** đừng viết — hãy *học* nó: cho một mạng thứ hai làm bộ phân biệt và huấn luyện bộ sinh để đánh lừa nó [2].
- Hệ quả** bộ phân biệt phạt bất cứ dấu hiệu nào lộ ra chất giả, kể cả dấu hiệu ta không nghĩ ra được, nên ảnh sắc nét [3].
- **cùng nguyên lý với C/17/02** — “không viết được hàm mất mát thì học nó” cũng chính là ý tưởng của mất mát tri giác, chỉ khác ở chỗ mạng chấm điểm được đóng băng thay vì huấn luyện đối kháng.
- Giá phải trả** mục tiêu không còn là cực tiểu hoá một hàm cố định mà là điểm cân bằng của một trò chơi hai người; “hội tụ” theo nghĩa thông thường không tồn tại, và mô hình có thể đạt mục tiêu bằng cách chỉ sinh ra một phần nhỏ của phân phối — *mode collapse*.
- Vấn đề mới** không có likelihood nên mất luôn công cụ chẩn đoán tự nhiên.
- 4 Vấn đề** muốn sắc nét như GAN nhưng ổn định như một bài hồi quy.
- Giải pháp** chẻ bài toán khó thành một chuỗi bài toán dễ: thêm nhiều dần cho tới khi ảnh thành nhiễu trắng, rồi học khử nhiễu ngược lại từng nấc [4], [5].
- Giả định** chuỗi đủ mịn để mỗi nấc chỉ cần một bước nhỏ.
- Hệ quả** hàm mất mát trở thành bình phương trung bình trên *nhiều* chứ không trên ảnh. Huấn luyện vì thế ổn định như mọi bài hồi quy. Đó là lý do diffusion đánh bại GAN: không phải bằng ý tưởng đẹp hơn, mà bằng việc dễ huấn luyện hơn ở quy mô lớn.
- Giá phải trả** sinh chậm vì phải chạy hàng chục tới hàng trăm nấc. Mục 17.3 dành trọn cho việc gỡ đúng cái giá này.

D. Ví dụ nhỏ nhất: vì sao mờ và mode collapse là hai mặt của một đồng xu

Lấy phân phối thật là hỗn hợp hai Gauss cách nhau: $p = \frac{1}{2}\mathcal{N}(-1, 0,25) + \frac{1}{2}\mathcal{N}(+1, 0,25)$. Bây giờ ép mô hình phải là *một* Gauss duy nhất $q = \mathcal{N}(\mu, \sigma^2)$. Mô hình quá nghèo để khớp — đúng tình huống thực tế, chỉ là ở chiều thấp.

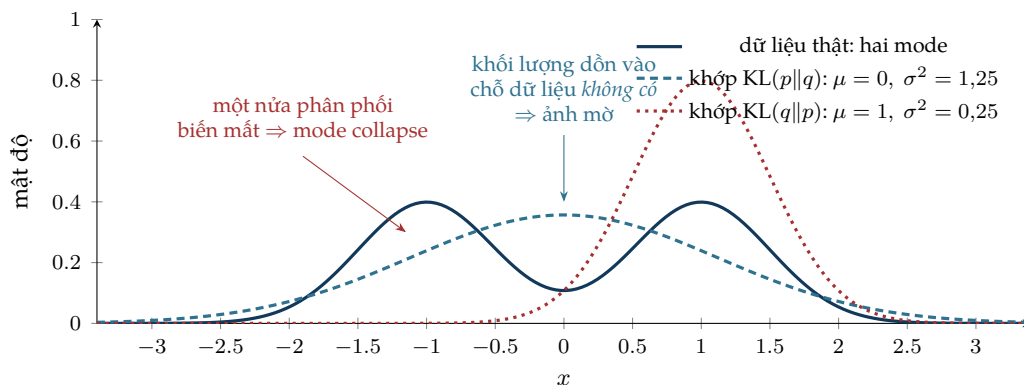
- **Nếu tối ưu $KL(p \parallel q)$** (hướng “thuận”, hướng mà cực đại hoá likelihood dùng): nghiệm là Gauss khớp mô-men của p , tức $\mu = 0$ và $\sigma^2 = 0,25 + 1 = 1,25$. Mô hình *trải rộng phủ cả hai mode*, và đặt nhiều khối lượng nhất ở $x = 0$ — đúng chỗ dữ liệu thật gần như không có gì. Dịch sang ảnh: đây chính là ảnh mờ.

- **Nếu tối ưu $KL(q \parallel p)$** (hướng “ngược”): phạt rất nặng việc đặt khối lượng ở nơi p nhỏ, nên nghiệm co về đúng một mode, $\mu \approx \pm 1, \sigma^2 \approx 0,25$. Mẫu sinh ra đều hợp lệ và sắc nét — nhưng một nửa phân phối biến mất. Dịch sang ảnh: đây chính là mode collapse.

Hai dòng trên là toàn bộ lời giải thích. VAE mờ và GAN thiếu đa dạng không phải hai lỗi cần sửa bằng hai mẹo khác nhau; chúng là hai nghiệm của cùng một bài toán, khác nhau ở hướng đo khoảng cách. Chi tiết về hai hướng KL nằm ở B/09-thiet-ke-ham-mat-mat; ở đây chỉ cần nhớ rằng chọn hàm mất mát là chọn kiểu thất bại.

↪ **tiền đề** B/09 — hai hướng của KL là điều kiện tiên quyết thật sự của cả chương: không nắm nó thì “mờ” và “mất mode” trông như hai lỗi rời rạc.

Hình 1.4 vẽ đúng ba phân phối của ví dụ trên.



Hình 1.4. Toàn bộ chương gói trong một hình. Cùng một phân phối thật hai mode, cùng một họ mô hình nghèo (một Gauss duy nhất), chỉ đổi *hướng* đo khoảng cách thì ra hai nghiệm trái ngược. Hướng thuận phạt việc bỏ sót mode nên nghiệm trải rộng và đặt nhiều khối lượng nhất ở $x = 0$, đúng chỗ dữ liệu thật gần như trống — dịch sang ảnh, đó là ảnh mờ của VAE. Hướng ngược phạt việc đặt khối lượng ở nơi mật độ thật thấp nên nghiệm co về đúng một mode — dịch sang ảnh, đó là mode collapse của GAN. Số trong hình tính tay được từ ví dụ ở mục D.

E. Giới hạn và trường hợp hỏng

Bảng và tam giác ở trên đơn giản hoá một bức tranh thật ra phức tạp hơn, và có ba chỗ cần nói thẳng:

- **Ranh giới giữa các họ đang mờ đi.** Diffusion trong không gian ẩn dùng một autoencoder kiểu VAE; *flow matching* là diffusion viết lại dưới dạng phương trình vi phân thường; mô hình sinh ảnh bằng token tự hồi quy đang quay lại mạnh mẽ. Bốn họ là bốn *nguyên lý*, không phải bốn hộp kín.
- **Likelihood cao không đồng nghĩa mẫu đẹp.** Một mô hình có thể đạt likelihood tốt nhờ mô hình hoá tốt các chi tiết tần số cao vô nghĩa mà vẫn sinh ra ảnh xấu; ngược lại GAN không có likelihood nhưng mẫu đẹp. Hai đại lượng đo hai thứ khác nhau.
- **Đánh giá là chỗ yếu nhất của cả mảng.** Nhìn vài chục mẫu được chọn lọc không nói được gì về độ phủ mode, mà độ phủ mới là thứ hay hỏng. Mục 17.4 nói kỹ về chỗ này.

CẠM BÃY

Đừng đánh giá một mô hình sinh bằng những mẫu đẹp nhất của nó. Chế độ hỏng đắt nhất — mode collapse và thu hẹp phân phối — *không nhìn thấy được* trong mẫu chọn lọc, vì các mẫu còn lại vẫn đẹp, chỉ là chúng giống nhau. Phép thử tối thiểu: sinh vài nghìn mẫu với cùng điều kiện rồi đo độ đa dạng (khoảng cách trung bình theo cặp trong không gian đặc

trưng, số cụm sau khi gom nhóm), và đối chiếu với cùng số lượng mẫu thật. Nếu bạn định dùng mẫu sinh làm dữ liệu huấn luyện thì phép thử này là bắt buộc chứ không phải tùy chọn — xem [C/17-mo-hinh-sinh/04-low-level-vision](#).

CẦN NHỚ

Mỗi họ mô hình sinh là một định nghĩa của “giống dữ liệu thật”, và *kiểu thất bại của mô hình được quyết định bởi định nghĩa đó, không phải bởi kiến trúc*. Trung bình theo pixel cho ảnh mờ; hướng ngược của KL cho mode collapse; chuỗi bài toán dễ cho huấn luyện ổn định nhưng lấy mẫu chậm. Biết mô hình tối ưu đại lượng gì là biết trước nó sẽ hỏng thế nào. [T2]

F. Kết nối

Mục này là khung cho cả chương 17: [C/17-mo-hinh-sinh/02-toi-uu-tren-pixel](#) tách riêng một ý tưởng cắt ngang cả bốn họ (đóng băng mạng, tối ưu trên chính ảnh); [C/17-mo-hinh-sinh/03-diffusion](#) đào sâu họ thứ tư và các bước biến nó thành sản phẩm; [C/17-mo-hinh-sinh/04-low-level-vision](#) cho thấy cùng bộ khái niệm này quay lại ở bài toán khôi phục ảnh. Hướng KL và hệ quả “mờ đổi lại thu hẹp” thuộc B/09-thiet-ke-ham-mat-mat. Ý tưởng “không viết được hàm mất mát thì học nó” cũng chính là ý tưởng của **mất mát tri giác** (*perceptual loss*) [6]. Nó cũng là ý tưởng của bộ phân biệt trong *pix2pix* [7], một cầu nối trực tiếp sang bài toán ảnh sang ảnh.

■ Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao ảnh mờ của VAE là *nghiệm đúng* của bài toán đã đặt ra chứ không phải dấu hiệu huấn luyện chưa đủ?
2. GAN không có likelihood. Điều đó lấy đi của bạn khả năng chẩn đoán cụ thể nào khi mô hình hoạt động kém?
3. Vì sao diffusion huấn luyện ổn định hơn GAN, và lý do đó có liên quan gì tới việc mục tiêu của nó là một bài hồi quy?
4. Bạn có một mô hình sinh và một trăm mẫu trông rất đẹp; phép thử ba mươi phút nào cho biết nó có bị thu hẹp phân phối hay không?
5. Nếu ràng buộc cứng của bạn là sinh 30 ảnh mỗi giây trên một GPU nhúng, tam giác ở Hình 1.2 chỉ về đâu, và bạn chấp nhận mất gì?

Nguồn

- [1] D. P. Kingma and M. Welling, “Auto-Encoding Variational Bayes,” in *ICLR*, 2014.
- [2] I. Goodfellow, J. Pouget-Abadie, M. Mirza, et al., “Generative Adversarial Nets,” in *NeurIPS*, 2014.
- [3] T. Karras, S. Laine, and T. Aila, “A Style-Based Generator Architecture for Generative Adversarial Networks,” in *CVPR*, 2019.
- [4] J. Ho, A. Jain, and P. Abbeel, “Denoising Diffusion Probabilistic Models,” in *NeurIPS*, 2020.
- [5] Y. Song, J. Sohl-Dickstein, D. P. Kingma, et al., “Score-Based Generative Modeling Through Stochastic Differential Equations,” in *ICLR*, 2021.
- [6] J. Johnson, A. Alahi, and L. Fei-Fei, “Perceptual Losses for Real-Time Style Transfer and Super-Resolution,” in *ECCV*, 2016.
- [7] P. Isola, J.-Y. Zhu, T. Zhou, et al., “Image-to-Image Translation with Conditional Adversarial Networks,” in *CVPR*, 2017.
- [8] L. A. Gatys, A. S. Ecker, and M. Bethge, “Image Style Transfer Using Convolutional Neural Networks,” in *CVPR*, 2016.
- [9] X. Huang and S. Belongie, “Arbitrary Style Transfer in Real-Time with Adaptive Instance Normalization,” in *ICCV*, 2017.
- [10] J. Adebayo, J. Gilmer, M. Muelly, et al., “Sanity Checks for Saliency Maps,” in *NeurIPS*, 2018.
- [11] G. Hinton, O. Vinyals, and J. Dean, “Distilling the Knowledge in a Neural Network,” in *NeurIPS Deep Learning Workshop*, 2015.
- [12] J. Song, C. Meng, and S. Ermon, “Denoising Diffusion Implicit Models,” in *ICLR*, 2021.
- [13] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, et al., “Flow Matching for Generative Modeling,” in *ICLR*, 2023.
- [14] J. Ho and T. Salimans, “Classifier-Free Diffusion Guidance,” *arXiv:2207.12598*, 2022.
- [15] R. Rombach, A. Blattmann, D. Lorenz, et al., “High-Resolution Image Synthesis with Latent Diffusion Models,” in *CVPR*, 2022.
- [16] W. Peebles and S. Xie, “Scalable Diffusion Models with Transformers,” in *ICCV*, 2023.
- [17] L. Zhang, A. Rao, and M. Agrawala, “Adding Conditional Control to Text-to-Image Diffusion Models,” in *ICCV*, 2023.
- [18] Y. Blau and T. Michaeli, “The Perception-Distortion Tradeoff,” in *CVPR*, 2018.
- [19] C. Ledig, L. Theis, F. Huszár, et al., “Photo-Realistic Single Image Super-Resolution Using a Generative Adversarial Network,” in *CVPR*, 2017.
- [20] Z. Wang, A. C. Bovik, H. R. Sheikh, et al., “Image Quality Assessment: From Error Visibility to Structural Similarity,” *IEEE TIP*, vol. 13, no. 4, pp. 600–612, 2004.
- [21] R. Zhang, P. Isola, A. A. Efros, et al., “The Unreasonable Effectiveness of Deep Features as a Perceptual Metric,” in *CVPR*, 2018.
- [22] M. Heusel, H. Ramsauer, T. Unterthiner, et al., “GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium,” in *NeurIPS*, 2017.

- [23] T. Brooks, B. Mildenhall, T. Xue, et al., “Unprocessing Images for Learned Raw Denoising,” in *CVPR*, 2019.
- [24] A. Abdelhamed, S. Lin, and M. S. Brown, “A High-Quality Denoising Dataset for Smartphone Cameras,” in *CVPR*, 2018.

1.2 Tối ưu trên pixel thay vì trên trọng số (Optimizing Pixels, Not Weights)

PHỤ THUỘC C/12-co-che-huan-luyen/01-co-hoc-cua-gradient (gradient chảy về đâu là do ta chọn); C/17-mo-hinh-sinh/01-bon-ho-mo-hinh-sinh (hàm mất mát quyết định kiểu thất bại); C/13-kien-truc-backbone/01-dong-cnn (bản đồ đặc trưng theo tầng).

TÓM TẮT

Có một ý tưởng đơn giản nối liền năm kỹ thuật trông chẳng liên quan gì tới nhau: đóng băng một mạng đã huấn luyện, coi *ảnh* là biến cần tối ưu, rồi hạ gradient trên chính pixel. Đối hàm mục tiêu là được một kỹ thuật khác — đảo ngược đặc trưng, DeepDream, trực quan hoá lớp, mẫu đối kháng, và chuyển phong cách. Đọc xong bạn viết được cả năm bằng cùng một vòng lặp mười dòng, và hiểu vì sao chuyển phong cách phiên bản tiến thẳng là ví dụ sạch nhất của mẫu hình “đẩy chi phí từ lúc chạy sang lúc huấn luyện”. Nếu chỉ nhớ một điều: tối ưu trên pixel tìm ra đúng những hướng mà mạng và mắt người *bất đồng*; khi hai bên tình cờ đồng ý ta gọi kết quả là nghệ thuật, khi bất đồng ta gọi nó là mẫu đối kháng — cùng một cơ chế.

KIẾN THỨC CẦN CÓ TRƯỚC

- **lan truyền ngược (backpropagation)** — thuật toán tính đạo hàm của một đại lượng đầu ra theo mọi nút của một đồ thị tính toán khả vi. Nó không phân biệt “tham số” với “dữ liệu” — đó là tiền đề của cả mục này.
- **bản đồ đặc trưng (feature map)** — tensor đầu ra của một tầng tích chập: một chồng “ảnh” nhỏ hơn, mỗi kênh phản ứng với một loại mẫu hình thị giác.
- **đóng băng (freeze)** — đặt cờ không tính gradient cho trọng số của một mạng, để nó chỉ còn đóng vai một hàm chấm điểm cố định.
- **ma trận Gram** — ma trận tích trong giữa các kênh của một bản đồ đặc trưng, cộng dồn trên mọi vị trí không gian; nó mô tả “kênh nào hay bật cùng kênh nào” mà bỏ hẳn thông tin vị trí.
- **mẫu đối kháng (adversarial example)** — ảnh bị thêm một nhiễu rất nhỏ, mắt người không thấy, nhưng đủ để mạng phân loại sai hoàn toàn.
- **hàm mục tiêu (objective function)** — toàn bộ đại lượng được tối ưu, gồm phần đo sai lệch (**hàm mất mát (loss function)**) cộng các số hạng chính quy hoá. Mục này dùng đúng từ “hàm mục tiêu” vì phần chính quy hoá ở đây quyết định kết quả không kém phần đo sai lệch.
- **chính quy hoá (regularization)** — phần cộng thêm vào hàm mục tiêu để loại bỏ các nghiệm “đúng về số nhưng vô lý về mặt ảnh”, ví dụ phạt biến thiên toàn phần để ép ảnh mượt.
- **feed-forward (tiến thẳng)** — chạy dữ liệu qua mạng đúng một lần từ đầu tới cuối, không lặp — đối lập với một vòng tối ưu nhiều trăm bước.

A. Đặt vấn đề

Trong huấn luyện thông thường, dữ liệu là hằng số và trọng số là ẩn số: ta cố định ảnh, tính gradient của hàm mất mát theo trọng số, rồi cập nhật trọng số. Nhưng **lan truyền ngược (backpropagation)** không quan tâm ta gọi cái gì là ẩn số — nó tính đạo hàm dọc theo một đồ thị khả vi, và đồ thị đó có đầu vào cũng là một nút. Vậy hoàn toàn có thể làm ngược lại: cố định trọng số, tính gradient theo *pixel đầu vào*, rồi cập nhật ảnh.

Câu hỏi thú vị không phải “làm được không” mà “để làm gì”. Câu trả lời: một mạng đã huấn luyện là một hàm chấm điểm ngầm định của thế giới thị giác — nó đã học được cái gì là mèo, cái gì là hoa văn, cái gì là biên. Tối ưu trên pixel là cách ép hàm chấm điểm đó *nói ra* thứ nó biết, dưới dạng một bức ảnh. Hình 1.5 là toàn bộ sự khác biệt, vẽ trong một hình.

Học mục này thì làm được gì mà trước đó không làm được? Bốn việc: cài chuyển phong cách, DeepDream, trực quan hoá lớp và tấn công đối kháng bằng cùng một đoạn code; giải thích được vì sao ma trận Gram bắt được “phong cách”; nhận ra mẫu hình dờn chỉ phí sang lúc huấn luyện khi gặp lại nó ở **chưng cất tri thức** (*knowledge distillation*); và có một công cụ chẩn đoán mạnh — muốn biết mạng của bạn thật sự nhìn cái gì, hãy tối ưu một ảnh làm nó phát điên rồi nhìn ảnh đó.



Hình 1.5. Cùng một đồ thị khả vi, hai cách dùng. Lan truyền ngược không phân biệt “dữ liệu” với “tham số” — nó chỉ tính đạo hàm dọc đồ thị; việc chọn nút nào là biến tối ưu hoàn toàn nằm ở người viết. Đây là lý do năm kỹ thuật ở Bảng 1.2 dùng chung một đoạn code.

↪ **trường hợp riêng của** C/12/01 — tối ưu trên pixel chỉ là lan truyền ngược chạy tới tận nút đầu vào thay vì dừng ở nút tham số.

TRỰC GIÁC

Hãy hình dung mạng đã huấn luyện là một giám khảo hội hoạ rất nhất quán nhưng không biết nói — ông ta chỉ chấm điểm, không giải thích. Tối ưu trên pixel là điều khắc một bức tượng bằng cách liên tục hỏi “sửa chỗ nào thì điểm tăng” rồi sửa đúng chỗ đó. Bức tượng cuối cùng phản ánh chính xác thứ mà giám khảo quan tâm — kể cả những thứ vô lý mà ông ta quan tâm nhưng con người thì không. Khi những thứ đó trùng với thẩm mỹ của ta, ta gọi kết quả là chuyển phong cách; khi chúng lệch hẳn, ta gọi kết quả là mẫu đối kháng.

B. Một vòng lặp, năm kỹ thuật

Vòng lặp chung ngắn tới mức viết ra được trọn vẹn:

```
# Đóng băng mạng; biến tối ưu là ảnh, không phải net.parameters()
for p in net.parameters():
    p.requires_grad_(False)
x = init_image.clone().requires_grad_(True)
opt = torch.optim.LBFGS([x])
def step():
    opt.zero_grad()
    loss = objective(net, x) # doi objective -> doi ky thuat
    loss.backward() # gradient chạy về tận x
    return loss
for _ in range(300):
    opt.step(step)
```

Chỉ dòng objective thay đổi giữa năm kỹ thuật ở Bảng 1.2. Đây không phải một sự trùng hợp thú vị mà là một bài học về cách đọc tài liệu: khi năm phương pháp chia chung một khung, học khung rẻ hơn học năm phương pháp gấp nhiều lần.

Bảng 1.2. Năm kỹ thuật cùng một vòng lặp, khác nhau đúng ở hàm mục tiêu.

Kỹ thuật	Hàm mục tiêu	Nó tiết lộ điều gì / hỏng khi nào
Đảo ngược đặc trưng	Khớp bản đồ đặc trưng của một ảnh mục tiêu ở một tầng	Tầng càng sâu càng mất thông tin vị trí; ảnh khôi phục ngày càng “đúng nội dung, sai bố cục”
DeepDream	Cực đại hoá chuẩn của activation ở một tầng	Cho thấy tầng đó thích hoa văn gì; kết quả lặp lại một mô-típ tới mức bệnh hoạn
Trực quan hoá lớp	Cực đại hoá điểm số của một lớp + phạt chính quy hoá	Không có phạt thì ra nhiều có điểm số cao — bằng chứng trực tiếp rằng mạng không nhìn như người
Mẫu đối kháng	Giảm điểm số lớp đúng với ràng buộc $\ \delta\ $ nhỏ	Tồn tại nhiều vô hình đối hản dự đoán; đây là chế độ hỏng của <i>mọi</i> mạng, không phải của một mạng kém
Chuyển phong cách	Nội dung ở tầng sâu + thống kê phong cách ở nhiều tầng nông	Phong cách nắm được là <i>kết cấu</i> , không phải bố cục hay ngữ nghĩa

C. Neural Style Transfer: vì sao ma trận Gram lại là “phong cách”

Bài toán của Gatys: giữ *nội dung* của ảnh A và *phong cách* của ảnh B [8]. Nội dung dễ định nghĩa — bản đồ đặc trưng ở một tầng sâu, vì tầng sâu giữ “có gì ở đâu” mà bỏ bớt chi tiết bề mặt. Phong cách mới là chỗ khó: nó phải là thứ mô tả “chất” của bức tranh mà không mô tả nội dung, tức là phải *độc lập với vị trí*.

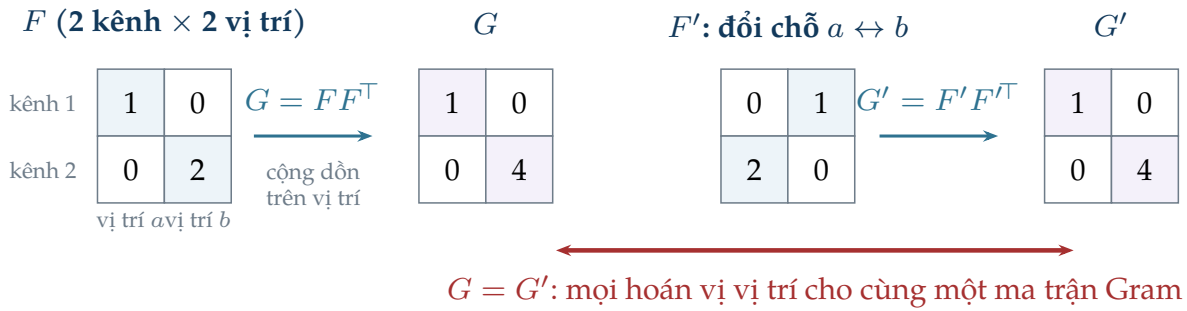
Lời giải là **ma trận Gram** (*Gram matrix*). Gọi $F \in \mathbb{R}^{C \times N}$ là bản đồ đặc trưng của một tầng, với C kênh và N vị trí không gian; đặt $G = FF^\top$, tức $G_{ij} = \sum_p F_{ip}F_{jp}$. Công thức này nói gì: G_{ij} đo mức độ hai kênh i và j cùng bật lên tại cùng một vị trí, cộng dồn trên toàn ảnh — nó là thống kê *đồng xuất hiện* của các đặc trưng, và phép cộng dồn trên p đã vứt bỏ hoàn toàn thông tin vị trí.

Tính tay trên ma trận 2×2 . Hai kênh, hai vị trí:

$$F = \begin{pmatrix} 1 & 0 \\ 0 & 2 \end{pmatrix} \Rightarrow G = FF^\top = \begin{pmatrix} 1 & 0 \\ 0 & 4 \end{pmatrix}, \quad F' = \begin{pmatrix} 0 & 1 \\ 2 & 0 \end{pmatrix} \Rightarrow G' = \begin{pmatrix} 1 & 0 \\ 0 & 4 \end{pmatrix}.$$

Đổi chỗ hai vị trí không gian mà G không đổi. Đó vừa là sức mạnh — “phong cách” đúng là thứ không phụ thuộc chỗ đặt — vừa là giới hạn: *Gram không phân biệt được bố cục*. Nó nắm được kết cấu nét cọ, bảng màu, độ thô của hạt; nó không nắm được “chân dung thì mất ở trên miệng”. Mọi thất bại quen thuộc của chuyển phong cách đều suy ra từ đây: khuôn mặt bị loang màu, bầu trời mọc nét cọ ở giữa mặt người, cấu trúc lớn của tranh gốc biến mất.

Hình 1.6 vẽ đúng phép tính đó.



Hình 1.6. Vì sao ma trận Gram bắt được “phong cách” — và vì sao nó không bao giờ bắt được bố cục. Phép cộng dồn trên các vị trí không gian biến F thành một thống kê *đồng xuất hiện giữa các kênh*, và phép cộng thì không quan tâm thứ tự. Hai bản đồ đặc trưng khác hẳn nhau về chỗ đặt cho ra cùng một G . Đó vừa là định nghĩa vừa là giới hạn của “phong cách”: nét cọ và bảng màu thì giữ được, “mắt nằm trên miệng” thì không.

Đáng dừng lại một nhịp ở chỗ này, vì đây là lần thứ hai ta gặp cùng một mẹo: G là một *tổng* trên các vị trí, tức một hàm đối xứng — đúng cấu trúc mà PointNet dùng khi lấy max trên các điểm (C/16-thi-giac-3d/02-bieu-dien-3d-va-depth). Hai lĩnh vực không liên quan, một nguyên lý: muốn bất biến với thứ tự hay vị trí thì gộp bằng một phép toán đối xứng, và cái giá luôn luôn là mất chính cái thông tin thứ tự đó.

↔ cùng nguyên lý với C/16/02 — PointNet lấy max trên tập điểm, Gram lấy tổng trên vị trí: cùng một hàm đối xứng, cùng một cái giá.

D. Dời chi phí từ lúc chạy sang lúc huấn luyện

Bản gốc của Gatys phải chạy vài trăm bước tối ưu cho *mỗi* ảnh, mất từ vài chục giây tới vài phút trên GPU — không dùng được cho video hay cho ứng dụng tương tác. Cách gỡ là một mẫu hình tổng quát, không riêng gì chuyển phong cách.

I. Mạch 2 — ba nấc của cùng một bài toán

1 Vấn đề

mỗi ảnh cần một vòng tối ưu riêng.

Giải pháp

huấn luyện một mạng **tiền thẳng** (*feed-forward*) để làm đúng việc mà vòng tối ưu làm, dùng chính hàm mất mát của Gatys làm tín hiệu giám sát — tức là **mất mát tri giác** (*perceptual loss*) [6].

Hệ quả

lúc chạy chỉ còn một lượt xuôi, nhanh hơn ba bậc độ lớn.

Giá phải trả

phải huấn luyện một mạng cho *mỗi* phong cách, và mất khả năng tinh chỉnh trọng số nội dung/phong cách sau khi huấn luyện.

2 Vấn đề

một mạng cho một phong cách là không mở rộng được.

Giải pháp

AdaIN: chuẩn hoá đặc trưng của ảnh nội dung về trung bình–độ lệch chuẩn 0–1 theo từng kênh, rồi *gán lại* trung bình và độ lệch chuẩn theo từng kênh của ảnh phong cách [9].

Giả định

điều đáng nói nằm ở giả định ngầm: nếu chỉ cần khớp thống kê bậc nhất theo kênh đã ra phong cách, thì phần lớn nội dung của ma trận Gram (thống kê bậc hai) là dư thừa — một phát hiện thực nghiệm khá bất ngờ.

Hệ quả

một mạng duy nhất làm được phong cách tùy ý, thời gian thực.

Giá phải trả

chất lượng thấp hơn bản tối ưu; các phong cách có cấu trúc lớn (tranh có hình khối rõ) bị mất nhiều nhất.

3 Vấn đề

ta vẫn chỉ có một đánh đổi chứ chưa có bữa trưa miễn phí.

Hệ quả

mẫu hình chung: độ khó không biến mất, nó chuyển từ lúc chạy sang lúc huấn luyện — và cùng mẫu hình đó sẽ xuất hiện ở chừng cất tri thức (D/20-toi-uu-va-nen-mo-hinh) và ở việc chừng cất bộ lấy mẫu diffusion (C/17-mo-hinh-sinh/03-diffusion).

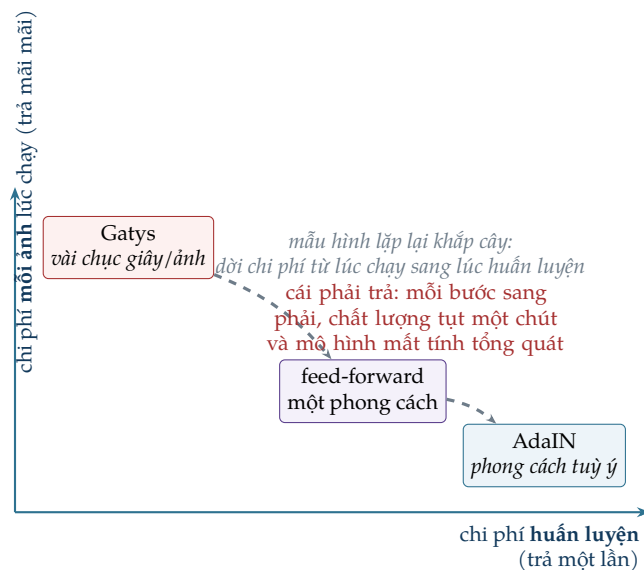
→ cùng nguyên lý với D/20 — chừng cất tri thức là chính mẫu hình này ở tầng mô hình: trả một lần lúc huấn luyện để khỏi trả mãi lúc suy luận.

→ cùng nguyên lý với C/16/03 — neural rendering làm đúng việc này trên một cảnh 3D thay vì trên một ảnh.

Bảng 1.3. Ba nấc của chuyển phong cách, đọc theo chỗ chi phí nằm ở đâu.

Cách làm	Chi phí mỗi ảnh	Chi phí huấn luyện	Hỏng khi nào
Gatys (tối ưu trên pixel)	Vài chục giây tới vài phút	Không có	Không dùng được cho video hay tương tác
Tiến thẳng một phong cách	Một lượt xuôi	Một lần huấn luyện cho mỗi phong cách	Số phong cách lớn; muốn đổi trọng số sau khi huấn luyện
AdaIN (phong cách tùy ý)	Một lượt xuôi	Một lần huấn luyện duy nhất	Phong cách có cấu trúc lớn; chất lượng dưới bản tối ưu

Hình 1.7 đặt ba nấc lên một mặt phẳng để thấy chi phí đã đi đâu.



Hình 1.7. Ba nấc của chuyển phong cách, vẽ trên mặt phẳng “ai trả chi phí”. Không nấc nào rẻ hơn nấc trước một cách miễn phí: cái được lấy đi ở trục dọc bị trả lại ở trục ngang, và một phần nhỏ nữa bị trả bằng chất lượng. Đây là cùng một hình sẽ đúng cho chừng cất tri thức và cho chừng cất bộ lấy mẫu diffusion — chỉ cần đổi nhãn ba hộp.

E. Giới hạn và trường hợp hỏng

Giới hạn nền tảng của cả mục này là ta đang tin vào một hàm chấm điểm mà chính ta không kiểm soát. Mạng đã huấn luyện có vô số hướng trong không gian ảnh mà nó nhạy còn mất người thì không — tối ưu trên pixel đi thẳng vào những hướng đó, vì gradient bao giờ cũng chỉ về hướng dốc nhất. Đó là lý do trực quan hoá lớp không có phạt chính quy hoá cho ra nhiều, và là lý do mẫu đối kháng tồn tại ở mọi mạng.

Hai chế độ hỏng cụ thể đáng nhớ:

- **Kết quả phụ thuộc mạnh vào khởi tạo và bộ tối ưu.** Bài toán không lỗi trên hàng trăm nghìn biến; khởi tạo từ ảnh nội dung, từ nhiễu, hay từ ảnh trắng cho ba kết quả khác nhau. Đây là tính chất của bài toán chứ không phải lỗi cài đặt — báo cáo kết quả mà không nói khởi tạo là báo cáo thiếu.
- **Chuyển phong cách mất tính nhất quán theo thời gian.** Áp cho từng khung hình video thì ảnh nhấp nháy, vì hai khung hình gần nhau rơi vào hai cực tiểu địa phương khác nhau. Cách chữa là thêm ràng buộc thời gian dựa trên dòng quang — và ta lại quay về đúng chủ đề của C/15-thi-giac-khong-thoi-gian.

CẠM BẮY

Đừng đọc một ảnh trực quan hoá lớp như bằng chứng “mạng hiểu khái niệm này”. Ảnh đó là nghiệm của một bài tối ưu có phạt chính quy hoá do bạn chọn, và phần lớn cấu trúc đẹp mắt trong đó đến từ chính phần phạt (biến thiên toàn phần, jitter, làm mượt) chứ không từ mạng. Bằng chứng: bỏ phạt đi thì ra nhiễu, dù điểm số của lớp còn cao hơn. Muốn dùng công cụ này để chẩn đoán, hãy luôn báo cáo phần phạt cùng với ảnh — xem thêm E/25-do-tin-cay-va-van-hanh về các phép kiểm tra tính đúng đắn của phương pháp giải thích [10].

CẦN NHỚ

Lan truyền ngược không phân biệt tham số với dữ liệu; chọn nút nào làm biến tối ưu là quyền của bạn. Đóng băng mạng và tối ưu trên pixel cho ra năm kỹ thuật khác nhau chỉ bằng cách đổi hàm mục tiêu — và bất cứ khi nào một vòng tối ưu lúc chạy quá đắt, hãy hỏi liệu có huấn luyện được một mạng làm thay nó hay không. Đó là mẫu hình dời chi phí sang lúc huấn luyện, gặp lại ở distillation. [T1]

F. Kết nối

Ý tưởng của mục này là bản sao chính xác của điều đã gặp ở C/16-thi-giac-3d/03-nerf-3dgs-va-hop-nhat: đặt thứ ta muốn vào hàm mất mát rồi hạ gradient trên chính đối tượng cần tạo ra — ở đó đối tượng là một cảnh 3D, ở đây là một ảnh. Ma trận Gram dùng chung nguyên lý hàm đối xứng với PointNet ở C/16-thi-giac-3d/02-bieu-dien-3d-va-depth. Mẫu đối kháng và trực quan hoá được nói kỹ ở E/25-do-tin-cay-va-van-hanh; mẫu hình dời chi phí sang lúc huấn luyện quay lại ở D/20-toi-uu-va-nen-mo-hinh [11] và ở phần chứng cất bộ lấy mẫu của C/17-mo-hinh-sinh/03-diffusion. Perceptual loss cũng chính là cầu nối sang C/17-mo-hinh-sinh/04-low-level-vision.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao cùng một đoạn code cho ra được cả một tác phẩm nghệ thuật lẫn một mẫu đối kháng — điều gì thực sự khác nhau giữa hai trường hợp?
2. Ma trận Gram bất biến với vị trí; hai thất bại nào của chuyển phong cách suy ra trực tiếp từ tính chất đó?
3. AdaIN chỉ khớp trung bình và độ lệch chuẩn theo kênh mà vẫn ra phong cách. Điều đó gợi ý gì về lượng thông tin thực sự cần thiết trong ma trận Gram?
4. Bạn thấy một ảnh trực quan hoá lớp rất đẹp trong một bài báo. Ba câu hỏi bạn phải hỏi trước khi tin nó là gì?

5. Mẫu hình “dời chi phí sang lúc huấn luyện” được mua bằng cái gì, và trong trường hợp cụ thể nào thì cái giá đó là không chấp nhận được?

Nguồn

- [1] D. P. Kingma and M. Welling, “Auto-Encoding Variational Bayes,” in *ICLR*, 2014.
- [2] I. Goodfellow, J. Pouget-Abadie, M. Mirza, et al., “Generative Adversarial Nets,” in *NeurIPS*, 2014.
- [3] T. Karras, S. Laine, and T. Aila, “A Style-Based Generator Architecture for Generative Adversarial Networks,” in *CVPR*, 2019.
- [4] J. Ho, A. Jain, and P. Abbeel, “Denoising Diffusion Probabilistic Models,” in *NeurIPS*, 2020.
- [5] Y. Song, J. Sohl-Dickstein, D. P. Kingma, et al., “Score-Based Generative Modeling Through Stochastic Differential Equations,” in *ICLR*, 2021.
- [6] J. Johnson, A. Alahi, and L. Fei-Fei, “Perceptual Losses for Real-Time Style Transfer and Super-Resolution,” in *ECCV*, 2016.
- [7] P. Isola, J.-Y. Zhu, T. Zhou, et al., “Image-to-Image Translation with Conditional Adversarial Networks,” in *CVPR*, 2017.
- [8] L. A. Gatys, A. S. Ecker, and M. Bethge, “Image Style Transfer Using Convolutional Neural Networks,” in *CVPR*, 2016.
- [9] X. Huang and S. Belongie, “Arbitrary Style Transfer in Real-Time with Adaptive Instance Normalization,” in *ICCV*, 2017.
- [10] J. Adebayo, J. Gilmer, M. Muelly, et al., “Sanity Checks for Saliency Maps,” in *NeurIPS*, 2018.
- [11] G. Hinton, O. Vinyals, and J. Dean, “Distilling the Knowledge in a Neural Network,” in *NeurIPS Deep Learning Workshop*, 2015.
- [12] J. Song, C. Meng, and S. Ermon, “Denoising Diffusion Implicit Models,” in *ICLR*, 2021.
- [13] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, et al., “Flow Matching for Generative Modeling,” in *ICLR*, 2023.
- [14] J. Ho and T. Salimans, “Classifier-Free Diffusion Guidance,” *arXiv:2207.12598*, 2022.
- [15] R. Rombach, A. Blattmann, D. Lorenz, et al., “High-Resolution Image Synthesis with Latent Diffusion Models,” in *CVPR*, 2022.
- [16] W. Peebles and S. Xie, “Scalable Diffusion Models with Transformers,” in *ICCV*, 2023.
- [17] L. Zhang, A. Rao, and M. Agrawala, “Adding Conditional Control to Text-to-Image Diffusion Models,” in *ICCV*, 2023.
- [18] Y. Blau and T. Michaeli, “The Perception-Distortion Tradeoff,” in *CVPR*, 2018.
- [19] C. Ledig, L. Theis, F. Huszár, et al., “Photo-Realistic Single Image Super-Resolution Using a Generative Adversarial Network,” in *CVPR*, 2017.
- [20] Z. Wang, A. C. Bovik, H. R. Sheikh, et al., “Image Quality Assessment: From Error Visibility to Structural Similarity,” *IEEE TIP*, vol. 13, no. 4, pp. 600–612, 2004.
- [21] R. Zhang, P. Isola, A. A. Efros, et al., “The Unreasonable Effectiveness of Deep Features as a Perceptual Metric,” in *CVPR*, 2018.
- [22] M. Heusel, H. Ramsauer, T. Unterthiner, et al., “GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium,” in *NeurIPS*, 2017.

- [23] T. Brooks, B. Mildenhall, T. Xue, et al., “Unprocessing Images for Learned Raw Denoising,” in *CVPR*, 2019.
- [24] A. Abdelhamed, S. Lin, and M. S. Brown, “A High-Quality Denoising Dataset for Smartphone Cameras,” in *CVPR*, 2018.

1.3 Diffusion và các bước làm cho nó khả thi (Diffusion Models)

PHỤ THUỘC C/17-mo-hinh-sinh/01-bon-ho-mo-hinh-sinh (vì sao diffusion ổn định hơn GAN); B/09-thiet-ke-ham-mat-mat (hồi quy trên nhiễu là bình phương trung bình); C/13-kien-truc-backbone (U-Net và transformer); C/12-co-che-huan-luyen (huấn luyện quy mô lớn).

TÓM TẮT

Diffusion biến một bài toán sinh rất khó thành một chuỗi bài toán khử nhiễu rất dễ, và đó là toàn bộ lý do nó thắng: hàm mất mát của nó là bình phương trung bình trên nhiễu, ổn định như mọi bài hồi quy. Mục này đi từ nguyên lý (quá trình thuận cố định, quá trình ngược học được, liên hệ với score function) tới bốn bước biến nó thành sản phẩm dùng được: bộ lấy mẫu ít bước, dẫn hướng không cần bộ phân lớp, khuếch tán trong không gian ẩn, và cơ chế điều khiển đầu ra. Đọc xong bạn biết mỗi tham số trong một pipeline sinh ảnh đang mua gì bằng gì — số bước, hệ số dẫn hướng, độ mạnh ảnh sang ảnh — và biết *latent diffusion* phá hỏng cái gì khi nó tiết kiệm tính toán. Nếu chỉ nhớ một điều: mọi cơ chế điều khiển trong diffusion đều mua độ trung thành bằng *độ đa dạng*, nên một pipeline chính tới mức đẹp nhất cũng là pipeline có phân phối hẹp nhất — điều nguy hiểm nếu bạn định dùng đầu ra làm dữ liệu huấn luyện.

KIẾN THỨC CẦN CÓ TRƯỚC

- **mô hình sinh (generative model)** — mô hình học phân phối của dữ liệu chứ không học một ánh xạ đầu vào → nhãn; dùng nó là lấy mẫu ra dữ liệu mới, không phải dự đoán.
- **nhiều Gauss (Gaussian noise)** — nhiễu cộng lấy từ phân phối chuẩn; “pha nhiễu” nghĩa là cộng thêm một lượng nhiễu như vậy vào ảnh, làm ảnh mờ dần thành hạt.
- **khử nhiễu (denoising)** — việc ước lượng lại ảnh sạch (hoặc lượng nhiễu đã cộng vào) từ ảnh đã bị pha nhiễu. Diffusion chỉ làm đúng việc này, lặp lại nhiều lần.
- **bộ lấy mẫu (sampler)** — đoạn mã quyết định đi từ nhiễu về ảnh theo mấy bước và mỗi bước dài bao nhiêu. Nó tách rời khỏi mô hình: cùng một mô hình đổi sampler cho ra ảnh khác nhau.
- **không gian ẩn (latent space)** — không gian chiều thấp mà một autoencoder nén ảnh vào; một ảnh 512×512 thành một tensor 64×64 , nên mọi phép tính sau đó rẻ đi hàng chục lần.
- **dẫn hướng (guidance)** — kỹ thuật kéo mẫu sinh ra về phía thoả mãn điều kiện (câu mô tả, ảnh cạnh...) mạnh hơn mức tự nhiên, đổi lại phân phối hẹp hơn.
- **score function (score function)** — gradient của log mật độ xác suất theo dữ liệu, $\nabla_x \log p(x)$; nó là một trường vector chỉ về hướng “đi đâu thì hợp lý hơn” tại mỗi điểm trong không gian ảnh.
- **SNR (signal-to-noise ratio)** — tỉ số giữa phần tín hiệu gốc còn lại và phần nhiễu ở một bước; SNR cao nghĩa là ảnh còn gần như nguyên vẹn.

A. Đặt vấn đề

Sau mục 17.1 ta có một yêu cầu rõ ràng: cần chất lượng của GAN mà không có sự bất ổn của một bài toán trò chơi. Ý tưởng của diffusion đơn giản một cách bất ngờ — nếu bài toán “sinh một ảnh từ hư không” quá khó, hãy chế nó thành một nghìn bài toán “làm ảnh này bớt nhiễu đi một chút”, mỗi bài dễ tới mức một mạng hồi quy bình thường giải được. Cộng đủ nhiều bước dễ thì ra một bước khó.

Cái phải trả cho sự chẻ nhỏ đó là thời gian: sinh một ảnh cần chạy mạng hàng chục tới hàng nghìn lần. Toàn bộ lịch sử kỹ thuật của mảng này, từ 2020 tới nay, là chuỗi các nỗ lực trả bớt cái giá đó mà không mất chất lượng — và đó cũng là cấu trúc của mục này.

Học mục này thì làm được gì mà trước đó không làm được? Đọc một tệp cấu hình `diffusers` và biết mỗi tham số đang đánh đổi gì; chẩn đoán được ảnh ra xấu là do bộ lấy mẫu, do hệ số dẫn hướng, hay do bộ mã hoá ẩn; và biết khi nào một mô hình sinh ảnh *không* nên được dùng để tạo dữ liệu huấn luyện.

TRỰC GIÁC

Hãy nghĩ tới một bức ảnh bị nhoè dần trong một chuỗi bản sao chép: bản 1 gần như nguyên vẹn, bản 500 lem nhem, bản 1000 chỉ còn là hạt. Việc đoán bản 1000 từ hư không là bất khả. Nhưng việc đoán bản 499 khi đã có bản 500 thì gần như tầm thường — chỉ phải lùi một nấc. Diffusion là phép nhận ra rằng nếu bạn làm được cái tầm thường đó một nghìn lần, bạn đã làm được cái bất khả. Và điều tinh tế: mấy nấc *cuối* (nhiều nhiều) là nơi mô hình quyết định bức ảnh sẽ vẽ *cái gì*, còn mấy nấc *đầu* (nhiều ít) chỉ tô nốt chi tiết.

B. Quá trình thuận, quá trình ngược

Quá trình thuận không có gì để học: nó chỉ là công thức pha nhiều Gauss dần dần, và nhờ tính chất cộng của Gauss, ta nhảy thẳng từ ảnh gốc tới bước bất kỳ mà không cần lặp:

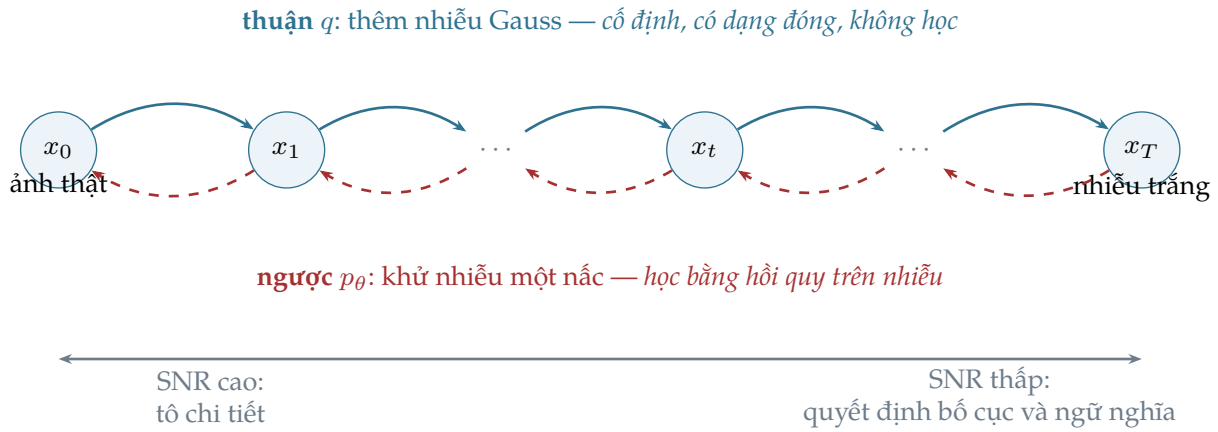
$$q(x_t | x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t) I), \quad \bar{\alpha}_t = \prod_{s \leq t} (1 - \beta_s).$$

Công thức này nói gì: ở bước t , ảnh còn lại đúng một phần $\sqrt{\bar{\alpha}_t}$ tín hiệu gốc, phần còn lại là nhiễu trắng — và vì có dạng đóng, ta lấy mẫu một t ngẫu nhiên rồi huấn luyện ngay, không phải chạy chuỗi. Đây là lý do kỹ thuật khiến diffusion huấn luyện được ở quy mô lớn.

Tính tay để thấy $T = 1000$ từ đâu ra. Với lịch β tuyến tính từ 10^{-4} tới 0,02, trung bình $\beta \approx 0,0101$, nên $\bar{\alpha}_T \approx \exp(-\sum_t \beta_t) = \exp(-10,1) \approx 4 \times 10^{-5}$, tức $\sqrt{\bar{\alpha}_T} \approx 0,006$. Nói cách khác, sau một nghìn bước thì chưa tới 1 % biên độ tín hiệu gốc còn sót lại — vừa đủ để coi x_T là nhiễu trắng thuần túy và bắt đầu quá trình ngược từ đó.

Quá trình ngược mới là phần học. Mạng nhận x_t và t , dự đoán *nhiều* đã thêm vào, và hàm mất mát là bình phương trung bình $\|\varepsilon - \varepsilon_\theta(x_t, t)\|^2$ [4]. Đây là điểm mấu chốt của cả họ mô hình: mục tiêu là một bài hồi quy tầm thường, không phải một trò chơi hai người, nên không có bất ổn, không có *mode collapse*, và tăng quy mô thì tốt lên đều đặn.

↪ **hệ quả của** B/09 — ổn định của diffusion đến thẳng từ dạng của hàm mất mát — bình phương trung bình trên nhiễu — chứ không từ kiến trúc nào.



Hình 1.8. Hai chiều của cùng một chuỗi. Chiều thuận không có tham số nào, nên toàn bộ khả năng của mô hình dồn vào chiều ngược. Trục dưới giải thích một điều thực hành quan trọng: các bước ở phía nhiễu nhiều quyết định *nội dung* ảnh, nên cắt bớt bước ở phía đó thay đổi ảnh nhiều hơn hẳn cắt bớt ở phía nhiễu ít — đó là lý do các lịch lấy mẫu ít bước không chia đều khoảng cách.

C. Vì sao gọi là “khớp score function”, và vì sao cần nhiều bước

Có một cách nhìn thứ hai cho cùng một mô hình, và nó giải thích được nhiều thứ hơn. Với phân phối p_t của dữ liệu đã pha nhiễu ở mức t , **score function** là $\nabla_x \log p_t(x)$ — vectơ chỉ về hướng “đi đâu thì hợp lý hơn”. Có thể chứng minh rằng mạng dự đoán nhiễu ở trên chính là ước lượng của score function sai khác một hệ số: $\varepsilon_\theta(x_t, t) \approx -\sqrt{1 - \alpha_t} \nabla_x \log p_t(x_t)$ [5]. *Nói gọn thì*: mô hình học một trường vectơ phủ khắp không gian ảnh, và lấy mẫu là thả một điểm nhiễu vào trường đó rồi để nó trôi về vùng có mật độ cao.

Cách nhìn này trả lời ngay câu hỏi “vì sao cần nhiều bước”: trôi theo một trường vectơ là giải một phương trình vi phân, và mỗi bước rời rạc là một xấp xỉ tuyến tính chỉ đúng trong lân cận nhỏ. Bước dài thì sai số rời rạc hoá lớn, ảnh ra méo. Từ đó suy ra hai hướng cải thiện, và cả hai đều đã thành chuẩn:

- **Đổi bộ giải.** DDIM biến quá trình lấy mẫu ngẫu nhiên thành một phương trình vi phân thường tất định, cho phép nhảy bước dài hơn với cùng mô hình đã huấn luyện [12]. Không huấn luyện lại gì cả — đây thuần tuý là giải số tốt hơn.
- **Làm thẳng quỹ đạo.** *Flow matching* huấn luyện thẳng một trường vận tốc nổi nhiễu với dữ liệu theo đường càng gần đường thẳng càng tốt [13]; quỹ đạo thẳng thì bộ giải cần ít bước hơn. Đây là hướng thiết kế thống trị các mô hình sinh lớn hiện nay.

D. Điều khiển đầu ra: dẫn hướng, không gian ẩn, ràng buộc cấu trúc

I Mạch 3 — bốn bước biến diffusion thành sản phẩm

1 Vấn đề

Giải pháp

mô hình sinh ra ảnh hợp lý nhưng không theo ý ta.

classifier-free guidance: trong lúc huấn luyện, thỉnh thoảng bỏ điều kiện đi để một mạng duy nhất học cả bản có điều kiện lẫn bản không; lúc lấy mẫu, ngoại suy $\tilde{\varepsilon} = \varepsilon_{\text{vô dk}} + w(\varepsilon_{\text{có dk}} - \varepsilon_{\text{vô dk}})$ [14]. Công thức này nói gì: lấy hiệu giữa “có điều kiện” và “không điều kiện” làm hướng của điều kiện, rồi đi quá lên w lần theo hướng đó.

Giá phải trả

$w > 1$ không phải là “làm đúng hơn” mà là *làm phân phối hẹp lại*: bám prompt tốt hơn, đa dạng kém đi, và ở w lớn thì màu bị cháy.

2 Vấn đề

chạy khuếch tán trên không gian pixel quá đắt.

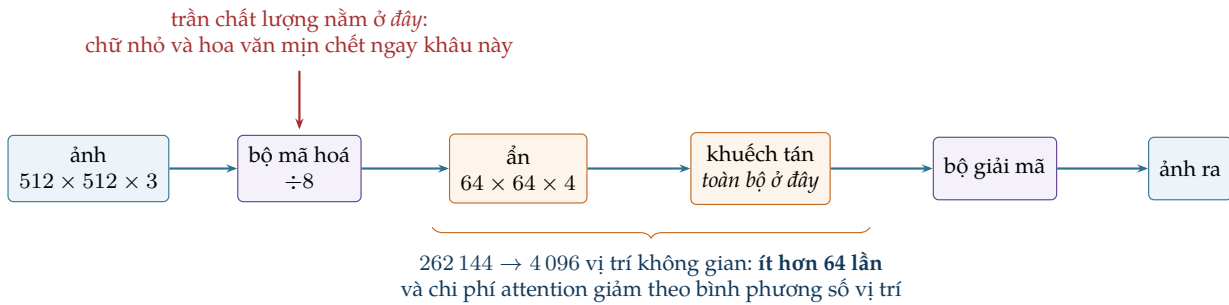
- Giải pháp** nén ảnh bằng một autoencoder rồi khuếch tán trong không gian ẩn [15]. Với hệ số nén 8, ảnh 512×512 còn 64×64 vị trí — ít hơn 64 lần, và vì chi phí *attention* tăng theo bình phương số vị trí, phần đó rẻ đi hàng nghìn lần.
- Hệ quả** đây chính là bước biến diffusion từ nghiên cứu thành sản phẩm chạy được trên GPU tiêu dùng.
- ↔ **ràng buộc bởi** B/11 — lợi ích của không gian ẩn tính trực tiếp từ ngân sách bộ nhớ và chi phí attention theo số vị trí.
- Giá phải trả** chất lượng bị chặn trên bởi chất lượng autoencoder; chi tiết rất nhỏ — chữ nhỏ, mắt người ở xa, hoa văn mịn — bị phá ngay ở khâu nén và không mô hình khuếch tán nào cứu lại được.
- 3 Vấn đề** U-Net khó mở rộng quy mô một cách có quy luật.
- Giải pháp** thay bằng *transformer* thao tác trên các *token* của không gian ẩn — *diffusion transformer* [16].
- Hệ quả** quy luật mở rộng quy mô trở nên dự đoán được, giống như ở mô hình ngôn ngữ.
- 4 Vấn đề** prompt bằng chữ không đủ để ràng buộc cấu trúc hình học.
- Giải pháp** *ControlNet*: nhân bản phần mã hoá của mô hình gốc, cho nhánh sao chép nhận điều kiện cấu trúc (cạnh, tư thế người, bản đồ độ sâu), và nối nó vào mô hình gốc qua các tầng tích chập *khởi tạo bằng 0* [17]. *Chi tiết khởi tạo 0 mới là chỗ hay*: ở bước huấn luyện đầu tiên nhánh mới đóng góp đúng bằng không, nên mô hình gốc không bị phá; huấn luyện chỉ *thêm* khả năng chứ không làm hỏng prior đã có.
- Giá phải trả** càng nhiều ràng buộc thì càng ít đa dạng — cùng một quy luật với dẫn hướng ở bước 1.

Bảng 1.4. Ba cách điều khiển đầu ra. Cả ba mua độ trung thành bằng cùng một đồng tiền là độ đa dạng.

Cách	Cơ chế	Ràng buộc được gì	Mất gì
Dẫn hướng (CFG)	Ngoại suy giữa nhánh có và không điều kiện	Ngữ nghĩa, độ bám prompt	Đa dạng; w lớn gây cháy màu
ControlNet	Nhánh sao chép, nối bằng tầng khởi tạo 0	Cấu trúc hình học: cạnh, tư thế, độ sâu	Tự do bố cục; thêm chi phí tính toán
Ảnh sang ảnh / inpainting	Bắt đầu từ ảnh đã pha nhiễu tới mức $t < T$	Bố cục và màu của ảnh gốc	Mức thay đổi bị chặn bởi t đã chọn

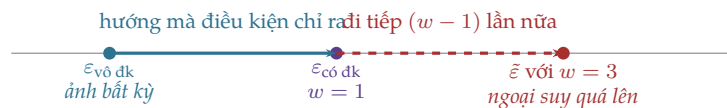
Riêng dòng cuối bảng đáng một câu giải thích, vì nó là hệ quả trực tiếp của Hình 1.8: tham số “độ mạnh” trong ảnh sang ảnh thực chất chỉ là *chọn bắt đầu từ nấc nào*. Bắt đầu ở nấc nhiễu ít thì ảnh gốc còn nguyên và mô hình chỉ tô lại bề mặt; bắt đầu ở nấc nhiễu nhiều thì bố cục cũng bị viết lại. Không có gì bí ẩn ở đây — chỉ là một chỉ số trên trục thời gian.

Hình 1.9 và Hình 1.10 vẽ hai bước quan trọng nhất trong mạch trên.



Hình 1.9. Latent diffusion, và chỗ nó lấy tiền về cũng như chỗ nó trả tiền ra. Toàn bộ phần dất — hàng chục tới hàng trăm lượt chạy mạng — được dờ vào không gian ẩn nhỏ hơn 64 lần về số vị trí, và đó là lý do một mô hình sinh ảnh chạy được trên GPU tiêu dùng. Cái giá nằm ở mũi tên đỏ: mọi chi tiết mà bộ mã hoá không giữ được thì không mô hình khuếch tán nào ở phía sau dựng lại được. Vì vậy phép chẩn đoán đầu tiên khi ảnh mất chi tiết là mã hoá rồi giải mã ngay ảnh gốc, bỏ qua khuếch tán.

càng sang phải: bám prompt càng chặt, phân phối càng **hẹp**, màu càng dễ cháy



Hình 1.10. Dẫn hướng không cần bộ phân lớp, nhìn như một phép ngoại suy trên đường thẳng. Hiệu giữa dự đoán có điều kiện và dự đoán không điều kiện chính là “hướng của điều kiện”; hệ số w quyết định đi bao xa theo hướng đó. $w = 1$ là dừng đúng dự đoán có điều kiện; $w > 1$ là đi *quá* lên. Điều quan trọng cần đọc ra từ hình: không có chỗ nào trong phép này làm mô hình “đúng hơn” — nó chỉ đẩy mẫu về vùng điển hình và cắt bớt đuôi, nên ảnh riêng lẻ đẹp hơn còn tập ảnh thì nghèo đi.

E. Giới hạn và trường hợp hỏng

- **Chậm là tính chất, không phải lỗi.** Mọi cách tăng tốc — ít bước, chưng cất bộ lấy mẫu, quỹ đạo thẳng — đều đang xấp xỉ cùng một quỹ đạo, và ở số bước rất nhỏ thì chất lượng và đa dạng đều tụt. Chưng cất bộ lấy mẫu là đúng mẫu hình “dời chi phí sang lúc huấn luyện” của mục 17.2.

↔ cùng nguyên lý với D/20 — cùng phép đổi chi phí suy luận lấy chi phí huấn luyện đã gặp ở chưng cất mô hình.

- **Trần chất lượng nằm ở autoencoder.** Nếu ảnh ra mất chữ nhỏ hoặc chi tiết mịn, đừng đổ cho mô hình khuếch tán trước khi kiểm tra: mã hoá rồi giải mã ngay ảnh gốc, không chạy khuếch tán, và xem chi tiết đó có sống sót không. Phép thử này mất ba mươi giây và loại bỏ được một nửa số giả thuyết.
- **Đếm, chữ, và quan hệ không gian vẫn yếu.** Đây là hệ quả của việc điều kiện đi vào mô hình qua *cross-attention* trên biểu diễn văn bản, chứ không qua một cơ chế suy luận rời rạc nào.
- **Lệch giữa mô hình và bộ lấy mẫu.** Một mô hình huấn luyện với lịch nhiễu này, chạy bằng bộ lấy mẫu giả định lịch khác, cho ra ảnh xấu mà không có thông báo lỗi nào. Đây là lỗi cấu hình phổ biến nhất trong thực hành.

CẠM BẦY

Tăng hệ số dẫn hướng cho tới khi ảnh “đẹp nhất” là một cái bẫy có hậu quả xa. Dẫn hướng mạnh không làm mô hình đúng hơn — nó *cắt bớt đuôi phân phối*, giữ lại phần lõi điển hình nhất. Ảnh riêng lẻ đẹp hơn, nhưng tập ảnh sinh ra nghèo hơn hẳn tập ảnh thật. Nếu bạn định dùng đầu ra làm dữ liệu huấn luyện, đây chính là cơ chế gây thu hẹp

phân phối qua từng vòng lặp: mỗi thể hệ học từ đuôi đã bị cắt của thể hệ trước. Xem [C/17-mo-hinh-sinh/04-low-level-vision](#).

↔ **tiền đề** [C/17/04](#) — muốn hiểu vì sao dữ liệu sinh có thể đầu độc mô hình sau, phải hiểu trước rằng mọi nóm đều khiến ở đây đều cắt đuôi phân phối.

CẦN NHỚ

Diffusion thắng không phải vì ý tưởng đẹp hơn GAN mà vì nó biến bài toán sinh thành một bài *hồi quy*, và hồi quy thì mở rộng quy mô được. Ba con số đáng nhớ khi chỉnh một pipeline: số bước quyết định sai số rời rạc hoá, hệ số dẫn hướng quyết định đánh đổi trung thành ↔ đa dạng, và hệ số nén của autoencoder quyết định trần chi tiết. Ba tham số đó độc lập nhau, và chẩn đoán sai lầm hay gặp nhất là đổ lỗi cho tham số thứ nhất khi thủ phạm là tham số thứ ba. [T2]

F. Kết nối

Mục này là phần chi tiết của họ thứ tư trong [C/17-mo-hinh-sinh/01-bon-ho-mo-hinh-sinh](#); “chưng cất bộ lấy mẫu” là bản sao của mẫu hình ở [C/17-mo-hinh-sinh/02-toi-uu-tren-pixel](#) và ở [D/20-toi-uu-va-nen-mo-hinh](#). Việc dùng đầu ra làm dữ liệu huấn luyện và hậu quả thu hẹp phân phối nằm ở [C/17-mo-hinh-sinh/04-low-level-vision](#); phần kiến trúc transformer thay U-Net nối về [C/13-kien-truc-backbone/03-vit-va-inductive-bias](#); chi phí bộ nhớ và tính toán của attention theo số vị trí nằm ở [B/11-gioi-han-tinh-toan](#).

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao quá trình thuận *không có tham số nào* lại là điều kiện kỹ thuật khiến diffusion huấn luyện được ở quy mô lớn?
2. Cách nhìn “score function” giải thích được điều gì mà cách nhìn “khử nhiễu từng nấc” không giải thích được?
3. Latent diffusion tiết kiệm tính toán bằng cách nào, và bạn thiết kế phép thử ba mươi giây nào để biết chi tiết bị mất là do khâu nén hay do khâu khuếch tán?
4. Vì sao tăng hệ số dẫn hướng làm ảnh đẹp hơn nhưng làm tập ảnh nghèo hơn — và khi nào điều đó là nguy hiểm chứ không chỉ là đáng tiếc?
5. Tầng tích chập nối của ControlNet được khởi tạo bằng 0. Bỏ chi tiết đó đi thì hỏng ở đâu, trong bước huấn luyện thứ mấy?
6. Theo Hình 1.8, nếu chỉ được giữ 10 trong 1000 bước, bạn đặt chúng ở đâu và vì sao?

Nguồn

- [1] D. P. Kingma and M. Welling, “Auto-Encoding Variational Bayes,” in *ICLR*, 2014.
- [2] I. Goodfellow, J. Pouget-Abadie, M. Mirza, et al., “Generative Adversarial Nets,” in *NeurIPS*, 2014.
- [3] T. Karras, S. Laine, and T. Aila, “A Style-Based Generator Architecture for Generative Adversarial Networks,” in *CVPR*, 2019.
- [4] J. Ho, A. Jain, and P. Abbeel, “Denoising Diffusion Probabilistic Models,” in *NeurIPS*, 2020.
- [5] Y. Song, J. Sohl-Dickstein, D. P. Kingma, et al., “Score-Based Generative Modeling Through Stochastic Differential Equations,” in *ICLR*, 2021.
- [6] J. Johnson, A. Alahi, and L. Fei-Fei, “Perceptual Losses for Real-Time Style Transfer and Super-Resolution,” in *ECCV*, 2016.
- [7] P. Isola, J.-Y. Zhu, T. Zhou, et al., “Image-to-Image Translation with Conditional Adversarial Networks,” in *CVPR*, 2017.
- [8] L. A. Gatys, A. S. Ecker, and M. Bethge, “Image Style Transfer Using Convolutional Neural Networks,” in *CVPR*, 2016.
- [9] X. Huang and S. Belongie, “Arbitrary Style Transfer in Real-Time with Adaptive Instance Normalization,” in *ICCV*, 2017.
- [10] J. Adebayo, J. Gilmer, M. Muelly, et al., “Sanity Checks for Saliency Maps,” in *NeurIPS*, 2018.
- [11] G. Hinton, O. Vinyals, and J. Dean, “Distilling the Knowledge in a Neural Network,” in *NeurIPS Deep Learning Workshop*, 2015.
- [12] J. Song, C. Meng, and S. Ermon, “Denoising Diffusion Implicit Models,” in *ICLR*, 2021.
- [13] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, et al., “Flow Matching for Generative Modeling,” in *ICLR*, 2023.
- [14] J. Ho and T. Salimans, “Classifier-Free Diffusion Guidance,” *arXiv:2207.12598*, 2022.
- [15] R. Rombach, A. Blattmann, D. Lorenz, et al., “High-Resolution Image Synthesis with Latent Diffusion Models,” in *CVPR*, 2022.
- [16] W. Peebles and S. Xie, “Scalable Diffusion Models with Transformers,” in *ICCV*, 2023.
- [17] L. Zhang, A. Rao, and M. Agrawala, “Adding Conditional Control to Text-to-Image Diffusion Models,” in *ICCV*, 2023.
- [18] Y. Blau and T. Michaeli, “The Perception-Distortion Tradeoff,” in *CVPR*, 2018.
- [19] C. Ledig, L. Theis, F. Huszár, et al., “Photo-Realistic Single Image Super-Resolution Using a Generative Adversarial Network,” in *CVPR*, 2017.
- [20] Z. Wang, A. C. Bovik, H. R. Sheikh, et al., “Image Quality Assessment: From Error Visibility to Structural Similarity,” *IEEE TIP*, vol. 13, no. 4, pp. 600–612, 2004.
- [21] R. Zhang, P. Isola, A. A. Efros, et al., “The Unreasonable Effectiveness of Deep Features as a Perceptual Metric,” in *CVPR*, 2018.
- [22] M. Heusel, H. Ramsauer, T. Unterthiner, et al., “GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium,” in *NeurIPS*, 2017.

- [23] T. Brooks, B. Mildenhall, T. Xue, et al., “Unprocessing Images for Learned Raw Denoising,” in *CVPR*, 2019.
- [24] A. Abdelhamed, S. Lin, and M. S. Brown, “A High-Quality Denoising Dataset for Smartphone Cameras,” in *CVPR*, 2018.

1.4 Low-level vision và trade-off không thể tránh (Low-Level Vision)

PHỤ THUỘC C/17-mo-hinh-sinh/01-bon-ho-mo-hinh-sinh (vì sao loss trung bình cho ảnh mờ); A/02-vat-ly-tao-anh/01-anh-sang-va-nhieu (nhiều cảm biến thật khác nhiều Gauss thế nào); B/07-chia-du-lieu-va-danh-gia (chọn chỉ số là chọn kết luận).

TÓM TẮT

Khử nhiễu, siêu phân giải, khử nhoè và HDR đều là bài toán *ill-posed*: nhiều ảnh đầu ra khác nhau cùng khớp với đầu vào, nên vai trò thật của mô hình là *chọn một* chứ không phải *tìm ra đáp án*. Từ đó suy ra một kết quả có chứng minh: không thể đồng thời tối ưu sai lệch theo pixel và độ chân thực thị giác. Vậy nên mọi tranh cãi “phương pháp nào tốt hơn” thật ra là tranh cãi về việc chọn điểm nào trên đường đánh đổi. Đọc xong bạn đọc được một bảng kết quả khôi phục ảnh mà không bị con số dẫn dắt, và biết khi nào một mô hình “đẹp hơn” thực chất đang bị. Nếu chỉ nhớ một điều: tối ưu PSNR cho ảnh mờ nhưng an toàn, còn tối ưu độ chân thực cho ảnh sắc nét nhưng có thể thêm chi tiết không hề có trong đầu vào. Trong ảnh y tế, giám định hay đo đạc, đó là lỗi đúng sai chứ không phải chuyện thẩm mỹ.

KIẾN THỨC CẦN CÓ TRƯỚC

- **low-level vision** — nhóm bài toán biến ảnh thành ảnh mà không cần hiểu nội dung: khử nhiễu, tăng độ phân giải, khử nhoè, ghép dải sáng động. Đối lập với high-level vision (phân loại, phát hiện) vốn biến ảnh thành nhãn.
- **ill-posed (đặt không chỉnh)** — bài toán mà nghiệm không tồn tại duy nhất hoặc không phụ thuộc liên tục vào dữ liệu; ở đây luôn là trường hợp “nhiều nghiệm cùng hợp lệ”, vì phép làm hỏng ảnh đã xoá mất thông tin.
- **toán tử thuận (forward operator)** — mô hình toán của quá trình làm hỏng ảnh — lấy trung bình rồi lấy mẫu thưa (siêu phân giải), chụp với nhân nhoè (khử nhoè), cộng nhiễu (khử nhiễu). Biết nó là biết mình đang phải đảo ngược cái gì.
- **prior (prior)** — giả định về “ảnh thật trông thế nào” dùng để chọn một nghiệm trong vô số nghiệm hợp lệ; ngày nay prior thường nằm ẩn trong trọng số của một mạng đã học.
- **PSNR** — sai lệch theo pixel quy ra decibel, suy ra trực tiếp từ bình phương trung bình; đo “gần theo nghĩa số học”.
- **SSIM** — chỉ số so sánh độ sáng, tương phản và cấu trúc cục bộ giữa hai ảnh; gần thị giác hơn PSNR nhưng vẫn thuộc nhóm đo sai lệch theo cặp.
- **LPIPS** — khoảng cách giữa hai ảnh đo trong không gian đặc trưng của một mạng đã huấn luyện; bám thị giác người tốt hơn, nhưng cần có ảnh gốc để so cặp.
- **FID** — khoảng cách giữa *phân phối* ảnh sinh và phân phối ảnh thật. Nó khớp một Gauss trên đặc trưng của một mạng, cần hàng vạn mẫu, và không nói gì về từng ảnh riêng lẻ.

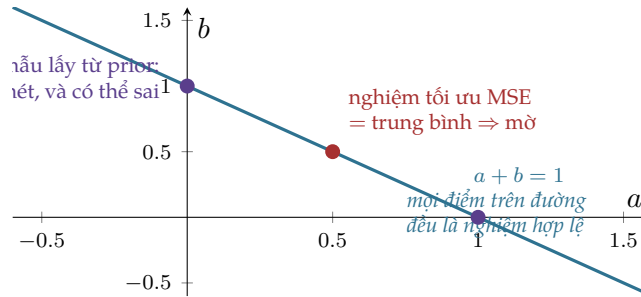
A. Đặt vấn đề

Mọi bài toán trong mục này có cùng một hình dạng: một toán tử thuận đã xoá bớt thông tin, và ta phải đi ngược. Điều quan trọng là thông tin đã mất thì *mất thật* — không thuật toán nào lấy lại được, chỉ có thể đoán.

Ví dụ nhỏ nhất, tính tay được. Lấy tín hiệu một chiều và giảm độ phân giải hai lần bằng cách lấy trung bình từng cặp: hai giá trị a, b cho ra một giá trị $m = (a + b)/2$. Bây giờ đi ngược: biết $m = 0,5$, hãy tìm a và b . Một phương trình, hai ẩn — nghiệm là cả một đường thẳng $\{(a, 1 - a)\}$. Cặp $(0,5; 0,5)$ hợp lệ, mà $(0; 1)$ cũng hợp lệ, và $(-3; 4)$ cũng vậy. Với siêu phân giải $\times 4$ trên ảnh, mỗi pixel thấp cấp ứng với 16 pixel cao cấp: một ràng buộc, mười sáu ẩn. Mô hình không giải bài toán này; nó chọn một điểm trên không gian nghiệm, và cái quyết định điểm nào là prior.

→ cùng nguyên lý với C/16/01 — thị giác 3D đối mặt đúng tình thế này: khi ràng buộc hình học không đủ, prior học được chọn nghiệm — và cũng mất luôn khả năng tự phát hiện sai.

Hình 1.11 vẽ đúng không gian nghiệm đó.



Hình 1.11. Bài toán ill-posed nhỏ nhất có thể vẽ được. Giảm độ phân giải hai lần bằng phép lấy trung bình biến (a, b) thành một số $m = 0,5$; đi ngược thì một phương trình phải xác định hai ẩn, nên nghiệm là cả một đường thẳng. Mô hình không “giải” bài toán này — nó chọn một điểm trên đường. Chọn trọng tâm thì được điểm số sai lệch tốt nhất và được ảnh mờ; chọn một điểm lấy từ phân phối ảnh thật thì được ảnh sắc nét và chấp nhận rủi ro chọn nhầm. Siêu phân giải $\times 4$ trên ảnh chỉ là hình này với một ràng buộc và mười sáu ẩn.

Điều đó dẫn thẳng tới câu hỏi trung tâm của cả mục: chọn điểm nào? Nếu chọn trung bình của mọi nghiệm hợp lệ thì được điểm số sai lệch tốt nhất, nhưng trung bình của nhiều ảnh sắc nét lệch nhau là một ảnh mờ — đúng cơ chế đã gặp ở mục 17.1.

→ hệ quả của C/17/01 — ảnh mờ ở đây và ảnh mờ của VAE là cùng một phép tính: nghiệm tối ưu của bình phương trung bình là kỳ vọng có điều kiện.

Nếu chọn một nghiệm cụ thể lấy từ phân phối ảnh thật thì được ảnh sắc nét trông rất thật, nhưng nó có thể khác hẳn ảnh gốc ở chi tiết. Đây không phải một lựa chọn kỹ thuật mà là một đánh đổi có chứng minh.

Học mục này thì làm được gì mà trước đó không làm được? Đọc một bảng so sánh phương pháp khôi phục ảnh và biết ngay vì sao phương pháp có PSNR cao nhất lại trông tệ nhất; chọn chỉ số đánh giá phù hợp với việc mình định làm; và quyết định có được phép dùng ảnh sinh làm dữ liệu huấn luyện hay không, dựa trên lý do chứ không dựa trên cảm giác.

TRỰC GIÁC

Khôi phục ảnh giống việc đọc một lá thư bị nhoè mực. Chỗ nhoè có thể là chữ “hoa” hoặc chữ “hòa” — nét mực không còn phân biệt được nữa. Bạn có hai cách. Cách thứ nhất: viết chồng cả hai khả năng lên nhau thành một vệt mờ; ai đọc cũng thấy chỗ đó không rõ, nên không ai bị lừa. Cách thứ hai: chọn dứt khoát một chữ và viết thật đẹp; lá thư đọc trôi chảy, trông như nguyên bản — và người đọc không có cách nào biết chữ đó là bạn đoán. Tối ưu PSNR là cách thứ nhất; tối ưu độ chân thực là cách thứ hai. Không có cách thứ ba.

B. Bốn bài toán, một hình dạng

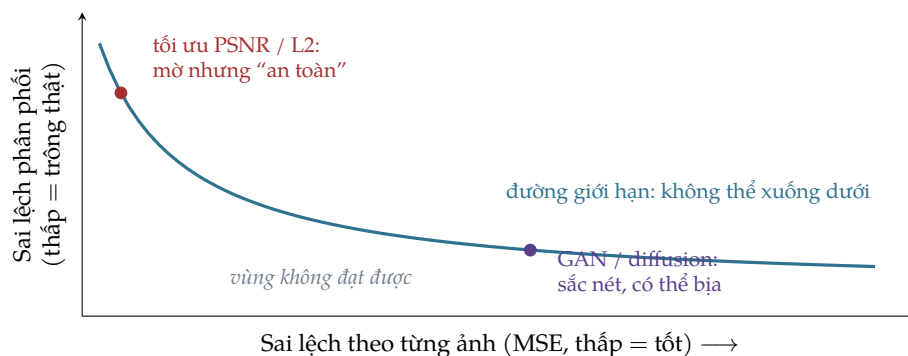
Bảng 1.5. Bốn bài toán low-level vision, đọc theo cái mà toán tử thuận đã xóa đi. Cột cuối cho biết prior phải bù vào chỗ nào — và do đó mô hình sẽ bịa ra loại nội dung gì khi nó sai.

Bài toán	Toán tử thuận	Cái bị xóa	Prior phải bù gì / hỏng khi
Khử nhiễu	Cộng nhiễu (thực tế phụ thuộc cường độ sáng, không phải Gauss đều)	Chi tiết biên độ nhỏ, lẫn vào nhiễu	Kết cấu tự nhiên; hỏng khi mô hình nhiễu lúc huấn luyện khác nhiễu thật
Siêu phân giải	Làm mờ rồi lấy mẫu thưa	Toàn bộ tần số cao trên ngưỡng Nyquist	Chi tiết tần số cao; hỏng ở chữ, khuôn mặt, biển số — những chỗ sai một chút là sai nghĩa
Khử nhoè	Chập với nhân nhoè (do chuyển động hoặc lệch nét)	Các tần số bị nhân nhoè triệt tiêu	Cấu trúc biên; hỏng khi nhân nhoè không đều trên ảnh hoặc không biết trước
HDR	Cắt ngưỡng trên và dưới của dải sáng	Thông tin ở vùng cháy sáng và vùng tối kịt	Nội dung vùng đã cháy; hỏng vì ở đó dữ liệu <i>bằng không</i> , không phải “nhiều”

Dòng cuối đáng chú ý vì nó là trường hợp cực đoan giúp nhìn rõ bản chất: ở vùng cháy sáng, cảm biến đã bão hoà nên mọi giá trị thật đều cho ra cùng một con số. Cái mô hình điền vào đó *hoàn toàn* là prior, không có một chút thông tin đo đạc nào. Ba bài toán còn lại chỉ là phiên bản nhẹ hơn của cùng tình huống.

C. Đánh đổi perceptual-distortion: một kết quả, không phải một kinh nghiệm

Đây là chỗ cần phân biệt rõ địa vị của các phát biểu. Trong mục 17.1, “thể lưỡng nan ba chiều” là một cách gọi tên quan sát thực nghiệm. Ở đây thì khác. Trước hết phải hình thức hoá hai đại lượng: *sai lệch* là kỳ vọng của một khoảng cách tính theo từng ảnh, còn *độ chân thực* là khoảng cách giữa phân phối ảnh khôi phục và phân phối ảnh thật. Với hai định nghĩa đó, người ta *chứng minh được* rằng hàm biểu diễn giới hạn giữa chúng là không tăng [18]. Nói bằng lời: muốn giảm sai lệch xuống dưới một mức nào đó thì buộc phải chấp nhận độ chân thực kém đi. Hình 1.12 vẽ tình thế đó.



Hình 1.12. Đánh đổi perceptual-distortion. Mọi phương pháp khôi phục ảnh đều nằm *trên* đường cong này; tiến bộ thuật toán đẩy đường cong xuống, còn việc chọn hàm mất mát chỉ là chọn *trượt dọc* theo nó. Hệ quả trực tiếp: không xếp hạng được các phương pháp bằng một con số duy nhất, và hai phương pháp ở hai đầu đường cong không so sánh được với nhau nếu chưa nói rõ mục đích sử dụng.

Hệ quả thực hành nghiêm túc hơn về ngoài của nó. Một mô hình siêu phân giải kiểu GAN hay diffusion [19] có thể dựng ra một khuôn mặt sắc nét, hợp lý và *sai người*, hoặc một biển số đọc

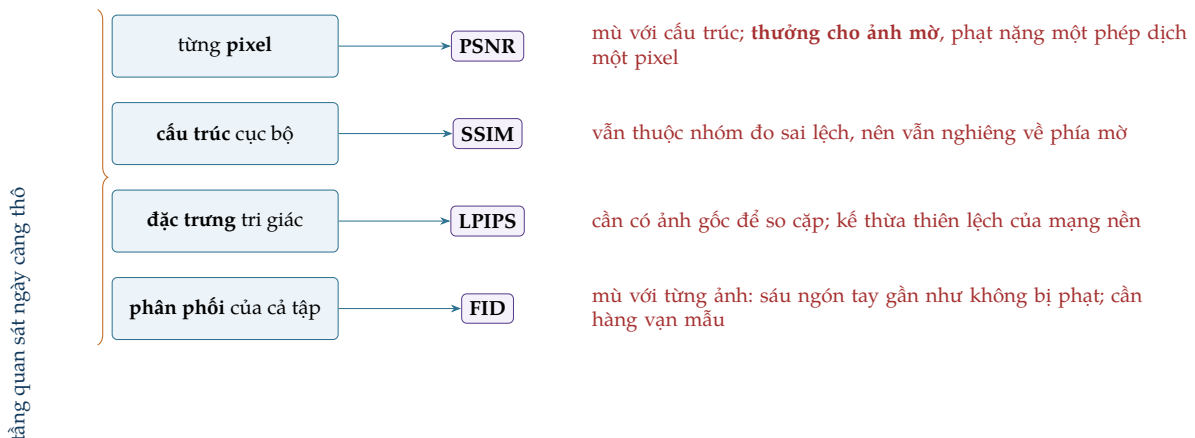
được và sai số. Trong ảnh giải trí, đó là tính năng. Trong ảnh y tế, giám định pháp y, đo đạc từ ảnh vệ tinh hay kiểm tra khuyết tật công nghiệp, đó là lỗi đúng sai. Nó còn nguy hiểm gấp bội vì đầu ra trông *đáng tin hơn* đầu vào.

D. Đánh giá: mỗi chỉ số nhạy với một thứ, và mù với thứ khác

Bảng 1.6. Bốn chỉ số hay dùng. Không chỉ số nào đủ một mình; chọn chỉ số là chọn trước loại lỗi mà bạn sẽ không nhìn thấy.

Chỉ số	Đo gì	Nhạy với	Mù với
PSNR	Bình phương trung bình theo pixel	Lệch sáng, lệch màu toàn cục, sai lệch biên độ lớn	Cấu trúc; nó <i>thường</i> cho ảnh mờ, và phạt nặng một phép dịch một pixel
SSIM	Tương quan cục bộ về sáng, tương phản, cấu trúc [20]	Mất cấu trúc cục bộ, nhoè	Sai lệch ngữ nghĩa; vẫn thuộc nhóm đo sai lệch nên vẫn nghiêng về ảnh mờ
LPIPS	Khoảng cách trong không gian đặc trưng học được [21]	Khác biệt mà mắt người thấy: kết cấu, nét	Cần ảnh gốc; kế thừa thiên lệch của mạng nền
FID	Khoảng cách giữa hai <i>phân phối</i> đặc trưng [22]	Lệch thống kê tổng thể, thu hẹp phân phối	Từng ảnh riêng lẻ; lỗi ngữ nghĩa (sáu ngón tay) gần như không bị phạt

Hình 1.13 xếp bốn chỉ số theo tầng mà chúng quan sát, kèm chỗ mù của từng cái.



Hình 1.13. Bốn chỉ số, bốn *tầng quan sát* khác nhau — và đó là lý do không chỉ số nào thay thế được chỉ số khác. Đi từ trên xuống, thước đo ngày càng ít quan tâm tới từng pixel và ngày càng quan tâm tới thống kê tổng thể; cột phải là cái mà mỗi tầng *không nhìn thấy*. Chọn chỉ số vì thế là chọn trước loại lỗi mà bạn sẽ để lọt, nên báo cáo tối thiểu phải có một chỉ số ở nhóm trên và một ở nhóm dưới.

Hai điểm thực hành ít được nói. Thứ nhất, FID cần cỡ hàng vạn mẫu mới ổn định. Giá trị của nó còn phụ thuộc mạnh vào chi tiết tiền xử lý: thư viện dùng để đổi kích thước ảnh, và chất lượng nén JPEG. Hệ quả là hai con số FID từ hai bài báo thường không so sánh trực tiếp được. Đó là lý do tồn tại các bản cài đặt chuẩn hoá như [GaParmar/clean-fid](#). Thứ hai, và quan trọng hơn: FID nhạy với việc *phân phối bị thu hẹp* — đúng thứ mà mắt người không phát hiện được khi nhìn từng ảnh. Đó là lý do nó vẫn là chỉ số chính dù mọi người đều biết nó có nhiều khuyết điểm.

E. Nhiều thật không phải nhiều Gauss

Một chế độ hỏng rất phổ biến, và hoàn toàn tránh được nếu biết trước. Gần như mọi mô hình khử nhiễu trong tài liệu được huấn luyện bằng cách cộng nhiễu Gauss trắng vào ảnh sạch, vì làm vậy thì có dữ liệu vô hạn miễn phí. Nhưng nhiễu của cảm biến thật là hỗn hợp của nhiễu Poisson phụ thuộc cường độ sáng và nhiễu đọc. Bộ xử lý ảnh trong máy còn biến đổi nó một cách phi tuyến qua khử khảm, cân bằng trắng, gamma và nén. Sau chuỗi đó, nhiễu trở nên có tương quan không gian và phụ thuộc màu [23]. Mô hình huấn luyện trên nhiễu Gauss đạt điểm rất cao trên bộ kiểm thử tổng hợp và tụt hẳn trên ảnh chụp thật [24].

Đây là một ca sách giáo khoa của lệch phân phối (B/08-dịch-chuyen-phan-phoi), và cách chữa cũng là cách chữa chuẩn: hoặc thu dữ liệu ghép cặp thật, hoặc mô phỏng đúng chuỗi vật lý bằng cách “gỡ ngược” quá trình xử lý ảnh về miền raw, thêm nhiễu đúng mô hình ở đó, rồi chạy xuôi trở lại. Chi tiết vật lý nằm ở A/02-vat-ly-tao-anh.

→ **ràng buộc bởi** A/02 — mô hình nhiễu dùng lúc huấn luyện phải khớp chuỗi vật lý thật của cảm biến, nếu không thì mọi con số đo được đều vô nghĩa.

→ **trường hợp riêng của** B/08 — nhiễu Gauss tổng hợp đối lại nhiễu cảm biến thật là một ca sách giáo khoa của lệch phân phối.

F. Dùng ảnh sinh làm dữ liệu huấn luyện

Đây là vấn đề mới của thời đại này, và câu trả lời không phải “có” hay “không” mà là một điều kiện.

- **Có ích khi** bạn dùng mô hình sinh để bù vào các vùng *hiếm* mà bạn biết trước là hiếm: tư thế lạ, thời tiết hiếm, khuyết tật ít gặp. Điều kiện đi kèm là bạn giữ được một tập kiểm thử *thật* hoàn toàn tách biệt để đo. Ở đây ảnh sinh đóng vai một dạng tăng cường dữ liệu có kiểm soát, và tín hiệu đánh giá vẫn đến từ thế giới thật.
- **Tự đầu độc khi** vòng lặp được lặp lại: mô hình sinh dữ liệu, dữ liệu huấn luyện mô hình sau, mô hình sau lại sinh dữ liệu. Cơ chế thu hẹp rất cụ thể. Mọi thao tác làm ảnh “đẹp hơn” — dẫn hướng mạnh, cắt ngưỡng, chọn lọc thủ công — đều cắt bớt đuôi của phân phối. Mỗi vòng lặp vì thế co phương sai lại một chút, và các mode hiếm biến mất hẳn. Hiện tượng này đã được chứng minh bằng thực nghiệm trong nhiều công trình gần đây; mức độ nghiêm trọng phụ thuộc rất mạnh vào một điều kiện: mỗi vòng có giữ lại dữ liệu thật hay không. Điểm này đáng nói rõ, vì các phát biểu kiểu “mô hình sẽ tự sụp đổ” thường bỏ qua nó.

G. Giới hạn và trường hợp hỏng

Giới hạn lớn nhất của cả mảng này là ta không có cách đánh giá tốt. Đường cong ở Hình 1.12 nói rằng một con số không đủ về mặt nguyên tắc, chứ không phải vì ta chưa nghĩ ra chỉ số hay hơn. Hệ quả là mọi bảng xếp hạng khôi phục ảnh đều mang một giả định ngầm về mục đích sử dụng, và giả định đó hầu như không bao giờ được viết ra.

Ba chế độ hỏng còn lại đáng nhớ: mô hình khử nhiễu huấn luyện ở một mức nhiễu cố định hỏng ở mức khác; mô hình siêu phân giải huấn luyện với một toán tử làm mờ cố định (thường là bicubic) hỏng ngay khi ảnh thật bị làm hỏng theo cách khác; và mọi mô hình trong nhóm này đều hỏng *im lặng* — đầu ra luôn là một ảnh trông hợp lý, không bao giờ là một thông báo “tôi không biết”.

CẠM BÃY

Cạm bẫy phổ biến nhất là dùng PSNR để chọn mô hình rồi ngạc nhiên vì mô hình được chọn trông tệ. Đây không phải trực trực mà là hành vi đúng của chỉ số: PSNR suy ra từ bình

phương trung bình, và nghiệm tối ưu của bình phương trung bình là trung bình của mọi ảnh hợp lệ, tức là ảnh mờ. *Chọn chỉ số là chọn trước kết luận.* Cách làm tối thiểu chấp nhận được có hai phần. Thứ nhất, báo cáo một chỉ số sai lệch và một chỉ số chân thực, tức là một ở mỗi đầu đường cong. Thứ hai, với ứng dụng có rủi ro, thêm một phép kiểm tra rằng mô hình *không* thêm nội dung. Phép thử rẻ nhất: cho đầu ra chạy lại qua toán tử thuận rồi so với đầu vào. Chỗ nào lệch là chỗ mô hình đã bịa.

CẦN NHỚ

Mọi bài toán low-level vision đều ill-posed, nên mô hình không “khôi phục” mà *chọn một nghiệm*, và prior là thứ quyết định chọn cái nào. Từ đó suy ra đánh đổi perceptual-distortion, vốn là một kết quả có chứng minh chứ không phải kinh nghiệm: sắc nét hơn đồng nghĩa với bịa nhiều hơn. Câu hỏi đúng khi chọn mô hình không phải “cái nào tốt hơn” mà “sai kiểu nào thì tôi chịu được”. [T1]

H. Kết nối

Cơ chế “loss trung bình cho ảnh mờ” ở đây là đúng cơ chế của VAE trong [C/17-mo-hinh-sinh/01-bon-ho-mo-hinh-sinh](#); các mô hình khôi phục hiện đại nhất chính là mô hình diffusion có điều kiện của [C/17-mo-hinh-sinh/03-diffusion](#), nên hệ số dẫn hướng ở đó cũng chính là nùm vặn trượt dọc đường cong ở đây. Vai trò của prior trong bài toán ill-posed là bản sao chính xác của vai trò prior trong [C/16-thi-giac-3d/01-bon-the-he-3d](#): cả hai lĩnh vực đều đòi ràng buộc cứng lấy tri thức học được, và đều mất khả năng tự phát hiện sai. Mô hình nhiễu thật nằm ở [A/02-vat-ly-tao-anh/01-anh-sang-va-nhieu](#); việc chọn chỉ số và giao thức đo nằm ở [B/07-chia-du-lieu-va-danh-gia](#); hậu quả vận hành của “hông im lặng” nằm ở [E/25-do-tin-cay-va-van-hanh](#).

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao một mô hình tối ưu MSE cho ảnh mờ, và vì sao đó là *ng nghiệm đúng* chứ không phải mô hình kém?
2. Đánh đổi perceptual-distortion khác về địa vị thế nào so với “thể lưỡng nan ba chiều” ở mục 17.1, và vì sao sự khác biệt đó quan trọng khi đọc paper?
3. FID nhạy với thứ gì mà mắt người không để ý, và mù với thứ gì mà mắt người nhận ra ngay?
4. Một mô hình khử nhiễu đạt kết quả xuất sắc trên bộ kiểm thử tổng hợp và kém hẳn trên ảnh điện thoại; cơ chế nào giải thích điều đó và bạn sửa bằng cách nào?
5. Trong trường hợp nào thì việc dùng ảnh sinh làm dữ liệu huấn luyện là an toàn, và điều kiện nào bị vi phạm thì nó trở thành tự đầu độc?
6. Nếu phải giao một mô hình siêu phân giải cho một hệ thống đọc biển số, bạn chọn điểm nào trên đường cong ở Hình 1.12, và bạn thêm phép kiểm tra nào để bảo vệ mình?

Nguồn

- [1] D. P. Kingma and M. Welling, “Auto-Encoding Variational Bayes,” in *ICLR*, 2014.
- [2] I. Goodfellow, J. Pouget-Abadie, M. Mirza, et al., “Generative Adversarial Nets,” in *NeurIPS*, 2014.
- [3] T. Karras, S. Laine, and T. Aila, “A Style-Based Generator Architecture for Generative Adversarial Networks,” in *CVPR*, 2019.
- [4] J. Ho, A. Jain, and P. Abbeel, “Denoising Diffusion Probabilistic Models,” in *NeurIPS*, 2020.
- [5] Y. Song, J. Sohl-Dickstein, D. P. Kingma, et al., “Score-Based Generative Modeling Through Stochastic Differential Equations,” in *ICLR*, 2021.
- [6] J. Johnson, A. Alahi, and L. Fei-Fei, “Perceptual Losses for Real-Time Style Transfer and Super-Resolution,” in *ECCV*, 2016.
- [7] P. Isola, J.-Y. Zhu, T. Zhou, et al., “Image-to-Image Translation with Conditional Adversarial Networks,” in *CVPR*, 2017.
- [8] L. A. Gatys, A. S. Ecker, and M. Bethge, “Image Style Transfer Using Convolutional Neural Networks,” in *CVPR*, 2016.
- [9] X. Huang and S. Belongie, “Arbitrary Style Transfer in Real-Time with Adaptive Instance Normalization,” in *ICCV*, 2017.
- [10] J. Adebayo, J. Gilmer, M. Muelly, et al., “Sanity Checks for Saliency Maps,” in *NeurIPS*, 2018.
- [11] G. Hinton, O. Vinyals, and J. Dean, “Distilling the Knowledge in a Neural Network,” in *NeurIPS Deep Learning Workshop*, 2015.
- [12] J. Song, C. Meng, and S. Ermon, “Denoising Diffusion Implicit Models,” in *ICLR*, 2021.
- [13] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, et al., “Flow Matching for Generative Modeling,” in *ICLR*, 2023.
- [14] J. Ho and T. Salimans, “Classifier-Free Diffusion Guidance,” *arXiv:2207.12598*, 2022.
- [15] R. Rombach, A. Blattmann, D. Lorenz, et al., “High-Resolution Image Synthesis with Latent Diffusion Models,” in *CVPR*, 2022.
- [16] W. Peebles and S. Xie, “Scalable Diffusion Models with Transformers,” in *ICCV*, 2023.
- [17] L. Zhang, A. Rao, and M. Agrawala, “Adding Conditional Control to Text-to-Image Diffusion Models,” in *ICCV*, 2023.
- [18] Y. Blau and T. Michaeli, “The Perception-Distortion Tradeoff,” in *CVPR*, 2018.
- [19] C. Ledig, L. Theis, F. Huszár, et al., “Photo-Realistic Single Image Super-Resolution Using a Generative Adversarial Network,” in *CVPR*, 2017.
- [20] Z. Wang, A. C. Bovik, H. R. Sheikh, et al., “Image Quality Assessment: From Error Visibility to Structural Similarity,” *IEEE TIP*, vol. 13, no. 4, pp. 600–612, 2004.
- [21] R. Zhang, P. Isola, A. A. Efros, et al., “The Unreasonable Effectiveness of Deep Features as a Perceptual Metric,” in *CVPR*, 2018.
- [22] M. Heusel, H. Ramsauer, T. Unterthiner, et al., “GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium,” in *NeurIPS*, 2017.

- [23] T. Brooks, B. Mildenhall, T. Xue, et al., “Unprocessing Images for Learned Raw Denoising,” in *CVPR*, 2019.
- [24] A. Abdelhamed, S. Lin, and M. S. Brown, “A High-Quality Denoising Dataset for Smartphone Cameras,” in *CVPR*, 2018.

1.5 Thực hành

Chương này có một đặc điểm hiếm: bài thực hành nhỏ nhất của nó — vẽ đường cong chất lượng theo số bước lấy mẫu — dạy được nhiều hơn cả chục trang lý thuyết, vì nó biến một đánh đổi trừu tượng thành một hình vẽ bạn tự tay tạo ra. Một lưu ý về bản rã trước khi vào danh sách: danh sách dưới đây chốt tại **tháng 9/2026**; tên các mô hình dẫn đầu sẽ cũ trong 6–12 tháng, trong khi phần hạ tầng — [diffusers](#), [k-diffusion](#), [clean-fid](#) — thì bền hơn nhiều — hãy đầu tư kỹ năng vào nhóm sau.

Repo và tài nguyên. *Hạ tầng:* [huggingface/diffusers](#), [comfyanonymous/ComfyUI](#), [crowsonkb/k-diffusion](#). *Ảnh:* FLUX.2 (dẫn về ảnh chân thực; bản [klein](#) 4B chạy được trong khoảng 8 GB VRAM), Stable Diffusion 3.5 (hệ sinh thái tinh chỉnh sâu nhất), Qwen-Image (dựng chữ trong ảnh), Z-Image Turbo (nhanh). *Video:* Wan 2.2 (Apache 2.0, lựa chọn an toàn nhất về giấy phép, bản 5B từ 8 GB), LTX-2.3 (sinh âm thanh đồng bộ trong cùng lượt khuếch tán), HunyuanVideo 1.5. *Học nguyên lý:* [lucidrains/denoising-diffusion-pytorch](#) — đọc mã nguồn này một lần đáng giá hơn đọc ba bài báo. *Đánh giá:* [GaParmar/clean-fid](#), [richzhang/PerceptualSimilarity](#).

Câu hỏi kiểm tra hiểu. Vì sao mô hình tối ưu MSE cho ảnh mờ, và perceptual loss “sửa” điều đó bằng cách đánh đổi gì? FID nhạy với thứ gì mà mắt người không quan tâm, và ngược lại? Latent diffusion tiết kiệm tính toán bằng cách nào, và mất gì ở đúng chỗ đó?

Mini project (vài giờ đến hai ngày). Cài một DDPM tối giản khoảng 200 dòng, huấn luyện trên MNIST hoặc CIFAR-10. Cố định mô hình, chỉ đổi bộ lấy mẫu và số bước (10, 25, 50, 250), rồi vẽ FID theo số bước và theo thời gian tường. Đường cong đó là bài học đánh đổi rõ nhất của cả mảng diffusion — và nó cũng cho bạn thấy tận mắt rằng chất lượng bão hòa trước khi thời gian bão hòa.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Độ hợp lý	Xác suất mà mô hình gán cho dữ liệu đã quan sát. Chỉ họ autoregressive tính được nó chính xác; GAN không có nó, và đó là lý do GAN mất công cụ chẩn đoán tự nhiên.
•• ELBO	Cận dưới của log độ hợp lý, thứ mà VAE thực sự tối ưu. Phần tái tạo của nó là bình phương trung bình theo pixel, nên ảnh mờ là hệ quả trực tiếp của chính công thức này.
••• Mode collapse	Mô hình chỉ sinh ra một phần nhỏ của phân phối trong khi vẫn đạt điểm cao theo tiêu chí huấn luyện. Nhìn mẫu chọn lọc không phát hiện được, vì các mẫu vẫn đẹp — chúng chỉ giống nhau.
••• Score matching	Huấn luyện bằng cách khớp gradient của log mật độ, $\nabla_x \log p(x)$, thay vì khớp mật độ. Đây là nền toán của diffusion: mô hình học một trường vectơ, và lấy mẫu là thả một điểm nhiều vào trường đó rồi để nó trôi.
••• Vì sao mô hình tối ưu MSE luôn cho ảnh mờ?	Vì nghiệm cực tiểu của bình phương trung bình là kỳ vọng có điều kiện, tức trung bình của mọi ảnh hợp lệ. Các ảnh đó lệch nhau vài pixel ở biên, nên trung bình của chúng nhoè.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●●● Vì sao diffusion huấn luyện ổn định hơn GAN?	Vì hàm mất mát của nó là bình phương trung bình trên nhiễu, tức một bài hồi quy tầm thường. GAN đi tìm điểm cân bằng của một trò chơi hai người, nên không có “hội tụ” theo nghĩa thông thường.
●● Ma trận Gram	Tích trong giữa các kênh của một bản đồ đặc trưng, cộng dồn trên mọi vị trí không gian. Phép cộng dồn xoá hết thông tin vị trí, nên nó giữ được kết cấu và bảng màu nhưng không giữ được bố cục.
●● AdaIN	Chuẩn hoá đặc trưng của ảnh nội dung về trung bình–độ lệch chuẩn 0–1 theo từng kênh, rồi gán lại thống kê của ảnh phong cách. Cho phép một mạng duy nhất làm phong cách tuỳ ý, đổi lại chất lượng thấp hơn bản tối ưu.
●● Vì sao tối ưu trên pixel lại tìm ra mẫu đối kháng?	Vì gradient luôn chỉ về hướng dốc nhất của hàm chàm điểm, mà mạng có vô số hướng nó nhảy còn mất người thì không. Tối ưu đi thẳng vào đúng những hướng bất đồng đó.
●●● Quá trình thuận	Phép pha nhiễu Gauss dần dần, không có tham số nào để học. Nhờ tính cộng của Gauss, ta nhảy thẳng từ ảnh gốc tới bước bất kỳ — đó là lý do kỹ thuật khiến diffusion huấn luyện được ở quy mô lớn.
●● Latent diffusion	Nén ảnh bằng một autoencoder rồi mới khuếch tán trong không gian ẩn. Với hệ số nén 8, ảnh 512×512 còn 64×64 vị trí — ít hơn 64 lần. Trữ chất lượng vì thế nằm ở autoencoder.
●● Classifier-free guidance	Ngoại suy giữa dự đoán có điều kiện và không điều kiện: $\tilde{\epsilon} = \epsilon_{\text{vô dk}} + w(\epsilon_{\text{có dk}} - \epsilon_{\text{vô dk}})$. Hệ số lớn bám câu mô tả chặt hơn, nhưng cắt bớt đuôi phân phối nên đa dạng giảm.
●● ControlNet	Nhánh sao chép phần mã hoá của mô hình gốc, nhận điều kiện cấu trúc (cạnh, tư thế, độ sâu) và nối vào qua các tầng khởi tạo bằng 0. Chi tiết khởi tạo 0 mới là phần đáng nhớ: nó khiến huấn luyện chỉ thêm khả năng chứ không phá prior đã có.
●● Vì sao “độ mạnh” chỉ là một chỉ số thời gian?	Vì nó quyết định bắt đầu khử nhiễu từ nấc nào. Bắt đầu ở nấc nhiễu ít thì mô hình chỉ tô lại bề mặt; bắt đầu ở nấc nhiễu nhiều thì bố cục cũng bị viết lại.
●●● Ill-posed	Bài toán có nhiều nghiệm cùng hợp lệ vì toán tử thuận đã xoá thông tin. Siêu phân giải $\times 4$ là một ràng buộc cho mười sáu ẩn; mô hình không giải mà <i>chọn</i> một nghiệm, và prior quyết định chọn cái nào.
●●● Vì sao không thể vừa sắc nét vừa đúng từng pixel?	Đó là đánh đổi perceptual–distortion: giảm sai lệch theo từng ảnh xuống dưới một mức thì buộc phải chấp nhận độ chân thực kém đi. Đây là kết quả <i>có chứng minh</i> dưới một cách hình thức hoá cụ thể, không phải lời khuyên rút ra từ kinh nghiệm.
●● FID	Khoảng cách giữa phân phối ảnh sinh và phân phối ảnh thật, ước lượng bằng cách khớp Gauss trên đặc trưng của một mạng. Nhạy với việc phân phối bị thu hẹp, mù với lỗi ngữ nghĩa của từng ảnh, và cần hàng vạn mẫu.
● Bản GAN sắc nét hơn hẳn với mắt người nhưng PSNR thua bản mờ — vì sao?	Vì hai bên nằm ở hai đầu đường cong đánh đổi. Bản mờ là trung bình của mọi ảnh hợp lệ nên tối ưu sai lệch theo pixel; bản GAN chọn dứt khoát một nghiệm nên sắc nét mà lệch từng pixel. Muốn thấy khác biệt thì phải đo thêm bằng LPIPS hoặc FID.
● Ảnh sinh mất chữ nhỏ và hoa văn mịn — chẩn đoán đầu tiên?	Mã hoá rồi giải mã ngay ảnh gốc, bỏ qua khuếch tán. Nếu chi tiết đã chết ở đó thì thủ phạm là autoencoder, không phải mô hình khuếch tán hay số bước lấy mẫu.
● Khử nhiễu rất tốt trên bộ tổng hợp, kém hẳn trên ảnh điện thoại — nguyên nhân?	Nó được huấn luyện trên nhiễu Gauss trắng, còn nhiễu thật là Poisson phụ thuộc cường độ sáng cộng nhiễu đọc, rồi bị bộ xử lý ảnh làm cho có tương quan không gian. Cách chữa là mô phỏng đúng chuỗi vật lý ở miền raw.

Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
diffusers, k-diffusion	Tách rời mô hình khỏi bộ lấy mẫu — đúng phân tách của mục 17.3, và là lý do đối sampler không cần huấn luyện lại	Hạ tầng bền; cũng là nơi lỗi lệch lịch nhiều hay xảy ra
Latent diffusion (dòng Stable Diffusion, FLUX)	Khuếch tán trong không gian ẩn để giảm số vị trí không gian; kiến trúc transformer thay U-Net	Trần chi tiết vẫn nằm ở autoencoder — kiểm tra bằng phép mã hoá rồi giải mã
ControlNet	Ràng buộc cấu trúc bằng nhánh sao chép nối qua tầng khởi tạo 0, để không phá prior đã học	Chi tiết khởi tạo 0 là phần đáng học nhất, không phải phần kiến trúc
Classifier-free guidance	Ngoại suy giữa nhánh có và không điều kiện; có mặt trong gần như mọi mô hình sinh ảnh hiện nay	Cũng là cơ chế cắt đuôi phân phối được nói ở mục 17.4
AdaIN	Chuẩn hoá rồi gán lại thống kê theo kênh — vẫn được dùng lại trong nhiều mạng sinh và mạng điều kiện hoá	Ý tưởng sống lâu hơn ứng dụng style transfer ban đầu của nó
clean-fid, LPIPS	Đánh giá ở hai tầng khác nhau (phân phối và theo cặp), đúng như bảng chỉ số của mục 17.4	Hạ tầng; tồn tại chính vì FID nhạy với chi tiết tiền xử lý

MINI PROJECT SLAM · Khử nhiễu trước frontend: “đẹp hơn” có nghĩa là “bám tốt hơn” không? · 1–2 ngày

Mục tiêu	nói thẳng: đây là liên hệ <i>gián tiếp</i> . Chương này về mô hình sinh, còn chỗ nó thật sự chạm vào visual SLAM là khâu tiền xử lý ảnh thiếu sáng đặt trước frontend. Mục tiêu là kiểm chứng bằng số một giả định mà hầu như ai cũng mặc nhiên tin: ảnh sau khử nhiễu trông sạch hơn thì frontend bám đặc trưng tốt hơn.
Dữ liệu	một chuỗi EuRoC thiếu sáng và nhiễu nhoè (V2_03 hoặc MH_04), cộng ba bản sao được làm tối và thêm nhiễu có kiểm soát để tạo một thang bốn mức nhiễu. Giữ nguyên chân trị (<i>ground truth</i>) của chuỗi gốc cho cả bốn mức.
Các bước	(1) chạy ORB-SLAM3 trên từng mức nhiễu <i>chưa</i> khử để lấy đường cơ sở: ATE, và số đặc trưng bám được trung bình mỗi khung hình; (2) chèn một bộ khử nhiễu nhẹ trước frontend — BM3D hoặc một CNN khử nhiễu nhỏ chạy đủ nhanh; (3) chạy lại với cấu hình y hệt, mỗi cấu hình ít nhất năm lần vì harness không tắt định; (4) với mỗi mức nhiễu ghi ba đại lượng: PSNR (hoặc LPIPS) của ảnh sau khử nhiễu, số đặc trưng bám được, và ATE trung bình cùng độ lệch chuẩn.
Xong khi	bạn có một bảng “mức nhiễu × {PSNR, số đặc trưng bám được, ATE}” và trả lời được bằng số: mức nhiễu nào thì khử nhiễu cải thiện ATE vượt độ lệch chuẩn của harness, mức nào thì không, và có tồn tại một mức mà PSNR <i>tăng</i> trong khi số đặc trưng bám được <i>giảm</i> hay không. Nếu có, hãy chỉ đúng chỗ đó — đó là đánh đổi perceptual-distortion hiện ra ở dạng cụ thể nhất.
Nối với	C/17/04 cho cơ chế: làm mượt để tối ưu sai lệch theo pixel cũng chính là xoá đi các góc mà detector cần; A/02 cho mô hình nhiễu cảm biến thật; E/24 và B/07 cho cách quyết định một chênh lệch có thật hay chỉ là nhiễu của harness. Dùng lại đúng harness lặp nhiều lần mà mini project chương 16 đã buộc bạn dựng.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- Vì nghiệm cực tiểu của bình phương trung bình là kỳ vọng có điều kiện — trung bình của *mọi* ảnh sắc nét khớp với đầu vào. Các ảnh đó lệch nhau vài pixel ở biên, nên trung bình của chúng là một ảnh mờ. Đây là nghiệm đúng của bài toán đã đặt ra, không phải dấu hiệu huấn luyện chưa đủ.
- Vì cả hai chỉ là hai hướng đo khoảng cách giữa phân phối mô hình và phân phối dữ liệu. Hướng thuận phạt việc bỏ sót mode nên mô hình trải rộng phủ cả hai mode và đặt khối lượng vào vùng

trống — ra ảnh mờ. Hướng ngược phát việc đặt khối lượng ở nơi dữ liệu hiếm nên mô hình co về một mode — ra mode collapse. Một lựa chọn, hai triệu chứng.

- A3. Vì hàm mất mát của nó là bình phương trung bình trên nhiều, tức một bài hồi quy tầm thường, trong khi GAN tối ưu điểm cân bằng của một trò chơi hai người. Hồi quy thì tăng quy mô là tốt lên đều đặn; trò chơi thì không có “hội tụ” theo nghĩa thông thường. Diffusion mua sự ổn định đó bằng số bước lấy mẫu.
- A4. Vì mọi nóm làm ảnh đẹp hơn — dẫn hướng mạnh, cắt ngưỡng, chọn lọc thủ công — đều hoạt động bằng cách cắt bớt đuôi phân phối. Từng ảnh đẹp hơn nhưng tập ảnh nghèo hơn, và nếu vòng lặp lặp lại thì mỗi thế hệ học từ đuôi đã bị cắt của thế hệ trước, phương sai co dần và các mode hiếm biến mất.

NÓI LẠI THẬT ĐƠN GIẢN

Tưởng tượng một cuộc thi vẽ, và ba giám khảo chấm cùng một bài. Người thứ nhất đặt bài thi chồng lên bản mẫu rồi đo xem mỗi nét lệch bao nhiêu milimét. Người thứ hai lùi ra xa ba mét, nheo mắt, và hỏi “nhìn tổng thể nó có ra dáng không”. Người thứ ba trộn bài đó vào một chồng năm trăm bài thật rồi xem có rút nó ra được không.

Ba cách chấm đó cho ba thí sinh thủ khoa hoàn toàn khác nhau, và đây mới là điều đáng nhớ: thí sinh sẽ luyện đúng theo cách mình bị chấm. Nếu bị chấm bằng thước, cách an toàn nhất là vẽ nhòe — một nét nhòe nằm giữa mọi khả năng thì không bị trừ nhiều ở bất cứ chỗ nào. Nếu bị chấm bằng mắt từ ba mét, cách thắng là vẽ mạnh tay và tự bịa thêm chi tiết cho sống động, kể cả những chi tiết không có trong bản mẫu.

Toàn bộ chương này nói rằng máy cũng vậy. Người ta không dạy máy “vẽ đẹp hơn”; người ta chỉ chọn một cách chấm, rồi máy tự tìm cách thắng cách chấm đó. Ảnh mờ không phải vì máy kém mà vì nó đang bị chấm bằng thước. Ảnh sắc nét nhưng bịa chi tiết không phải vì máy nói dối mà vì nó đang bị chấm bằng mắt.

Cái giá là bạn không thể chọn cả hai. Muốn không bao giờ bịa thì phải chấp nhận nhòe; muốn sắc nét thì phải chấp nhận rằng một phần những gì bạn thấy là do máy tự thêm vào. Trong ảnh trang trí, đó là ưu điểm. Trong ảnh y tế hay ảnh dùng làm bằng chứng, đó là sai lầm nghiêm trọng.

Chỗ không rút gọn được: câu “không thể chọn cả hai” ở trên. Nói bằng lời thường thì nó nghe như một lời khuyên rút ra từ kinh nghiệm, nhưng thực chất nó là một kết quả *chứng minh được* dưới một cách phát biểu chặt chẽ. Sự khác biệt đó rất quan trọng khi đọc một bài báo tuyên bố “chúng tôi vừa chính xác hơn vừa sắc nét hơn”: với một lời khuyên kinh nghiệm thì tuyên bố ấy có thể đúng, với một định lý thì nó chỉ có thể đúng nếu bài báo đang đo hai thứ khác nhau.

Một câu: Mọi mô hình sinh đều là một thí sinh luyện theo cách chấm điểm, nên đổi thước đo là đổi kiểu sai lầm, không phải đổi mức thông minh.

Kết nối

C/18-pretraining-va-foundation tiếp tục câu chuyện “tín hiệu giám sát đến từ đâu” mà chương này chạm vào khi nói về dữ liệu sinh; D/20-toi-uu-va-nen-mo-hinh trình bày đầy đủ mẫu hình dờ chi phí sang lúc huấn luyện; E/25-do-tin-cay-va-van-hanh biến “hông im lặng” của các mô hình khôi phục thành thiết kế phòng thủ cụ thể; và C/16-thi-giac-3d là chương song song, nơi cùng nguyên lý “prior chọn nghiệm” xuất hiện trong bài toán hình học.

Tổng kết chương

Ta đã đi qua bốn tầng của một câu hỏi duy nhất — khi nhiều đầu ra cùng hợp lệ thì cái gì chọn? — và câu trả lời lặp đi lặp lại là: hàm mất mát chọn, chứ không phải kiến trúc chọn. Bốn họ mô hình sinh khác nhau vì bốn cách viết “giống dữ liệu thật” thành một con số, và kiểu thất bại của mỗi họ suy ra được từ chính con số đó. Tối ưu trên pixel cho thấy điều này ở dạng thuần khiết nhất: cùng một vòng lặp, đổi hàm mục tiêu là được một kỹ thuật khác hẳn, từ tác phẩm nghệ thuật tới mẫu đối kháng. Diffusion thắng không nhờ ý tưởng đẹp hơn mà nhờ biến bài toán sinh

thành một bài hỏi quy có thể mở rộng quy mô. Và low-level vision cho thấy khi hậu quả là thật, cùng một đánh đổi đó trở thành câu hỏi “tôi chịu được kiểu sai nào”.

Ba chỗ còn bỏ ngỏ, đáng theo dõi: thứ nhất, việc rút số bước lấy mẫu xuống một chữ số mà không mất đa dạng — flow matching và chưng cất bộ lấy mẫu đang tiến nhanh nhưng chưa có lời giải sạch; thứ hai, đánh giá mô hình sinh, nơi mọi chỉ số hiện có đều mù ở một chỗ quan trọng và chưa ai có phương án thay thế thuyết phục; thứ ba, hệ quả dài hạn của việc dữ liệu sinh tràn vào tập huấn luyện của thể hệ mô hình sau — cơ chế thu hẹp phân phối đã rõ, nhưng độ nghiêm trọng trong thực tế còn phụ thuộc nhiều điều kiện chưa được đo đủ.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Câu hỏi chung của cả chương là “khi nhiều đầu ra cùng hợp lệ thì cái gì quyết định chọn cái nào”. Với VAE, GAN, diffusion và một mạng khôi phục ảnh huấn luyện bằng MSE, mỗi cái trả lời bằng cơ chế nào, và hai cái nào trong bốn thực ra đang đưa cùng một câu trả lời?
2. Mọi cơ chế làm ảnh đẹp hơn đều mua độ trung thành bằng độ đa dạng — hệ số dẫn hướng, loss cảm nhận và ma trận Gram đều thuộc họ đó. Ba cái này thu hẹp phân phối ở ba chỗ khác nhau nào trong pipeline?
3. Đánh đổi perceptual-distortion ở mục 4 và thể lưỡng nan ba chiều ở mục 1 nghe rất giống nhau. Chúng khác nhau ở địa vị lý thuyết thế nào, và vì sao gộp nhầm hai thứ đó lại dẫn tới kết luận sai khi đọc paper?
4. Mẫu đối kháng ở mục 2 và ảnh có FID rất tốt nhưng sai chi tiết ở mục 4 đều là “tối ưu đúng thước đo, sai mục đích”. Điểm chung về cơ chế là gì, và cùng một cách phòng vệ có dùng được cho cả hai không?
5. Bạn dùng đầu ra của một mô hình sinh làm dữ liệu huấn luyện cho mô hình khác. Điều kiện nào phải giữ để việc đó an toàn, và triệu chứng đầu tiên khi điều kiện bị vi phạm hiện ra ở đâu?
6. Latent diffusion, AdaIN và mẫu hình “dời chi phí sang lúc huấn luyện” đều đổi chất lượng lấy tốc độ. Với ràng buộc thời gian thực trên thiết bị nhúng, cái nào bị loại đầu tiên và vì sao?
7. Cả bốn mục đều cảnh báo về việc tin vào thứ nhìn thấy được — ảnh mẫu đẹp, bản đồ attention, ảnh trực quan hoá lớp, ảnh khử nhiễu sắc nét. Có quy tắc chung nào rút ra được không, và nó buộc bạn đo thêm cái gì?

Nguồn

- [1] D. P. Kingma and M. Welling, “Auto-Encoding Variational Bayes,” in *ICLR*, 2014.
- [2] I. Goodfellow, J. Pouget-Abadie, M. Mirza, et al., “Generative Adversarial Nets,” in *NeurIPS*, 2014.
- [3] T. Karras, S. Laine, and T. Aila, “A Style-Based Generator Architecture for Generative Adversarial Networks,” in *CVPR*, 2019.
- [4] J. Ho, A. Jain, and P. Abbeel, “Denoising Diffusion Probabilistic Models,” in *NeurIPS*, 2020.
- [5] Y. Song, J. Sohl-Dickstein, D. P. Kingma, et al., “Score-Based Generative Modeling Through Stochastic Differential Equations,” in *ICLR*, 2021.
- [6] J. Johnson, A. Alahi, and L. Fei-Fei, “Perceptual Losses for Real-Time Style Transfer and Super-Resolution,” in *ECCV*, 2016.
- [7] P. Isola, J.-Y. Zhu, T. Zhou, et al., “Image-to-Image Translation with Conditional Adversarial Networks,” in *CVPR*, 2017.
- [8] L. A. Gatys, A. S. Ecker, and M. Bethge, “Image Style Transfer Using Convolutional Neural Networks,” in *CVPR*, 2016.
- [9] X. Huang and S. Belongie, “Arbitrary Style Transfer in Real-Time with Adaptive Instance Normalization,” in *ICCV*, 2017.
- [10] J. Adebayo, J. Gilmer, M. Muelly, et al., “Sanity Checks for Saliency Maps,” in *NeurIPS*, 2018.
- [11] G. Hinton, O. Vinyals, and J. Dean, “Distilling the Knowledge in a Neural Network,” in *NeurIPS Deep Learning Workshop*, 2015.
- [12] J. Song, C. Meng, and S. Ermon, “Denoising Diffusion Implicit Models,” in *ICLR*, 2021.
- [13] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, et al., “Flow Matching for Generative Modeling,” in *ICLR*, 2023.
- [14] J. Ho and T. Salimans, “Classifier-Free Diffusion Guidance,” *arXiv:2207.12598*, 2022.
- [15] R. Rombach, A. Blattmann, D. Lorenz, et al., “High-Resolution Image Synthesis with Latent Diffusion Models,” in *CVPR*, 2022.
- [16] W. Peebles and S. Xie, “Scalable Diffusion Models with Transformers,” in *ICCV*, 2023.
- [17] L. Zhang, A. Rao, and M. Agrawala, “Adding Conditional Control to Text-to-Image Diffusion Models,” in *ICCV*, 2023.
- [18] Y. Blau and T. Michaeli, “The Perception-Distortion Tradeoff,” in *CVPR*, 2018.
- [19] C. Ledig, L. Theis, F. Huszár, et al., “Photo-Realistic Single Image Super-Resolution Using a Generative Adversarial Network,” in *CVPR*, 2017.
- [20] Z. Wang, A. C. Bovik, H. R. Sheikh, et al., “Image Quality Assessment: From Error Visibility to Structural Similarity,” *IEEE TIP*, vol. 13, no. 4, pp. 600–612, 2004.
- [21] R. Zhang, P. Isola, A. A. Efros, et al., “The Unreasonable Effectiveness of Deep Features as a Perceptual Metric,” in *CVPR*, 2018.
- [22] M. Heusel, H. Ramsauer, T. Unterthiner, et al., “GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium,” in *NeurIPS*, 2017.

- [23] T. Brooks, B. Mildenhall, T. Xue, et al., “Unprocessing Images for Learned Raw Denoising,” in *CVPR*, 2019.
- [24] A. Abdelhamed, S. Lin, and M. S. Brown, “A High-Quality Denoising Dataset for Smartphone Cameras,” in *CVPR*, 2018.